

```

import os
import cv2
import numpy as np
import tifffile as tiff
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F

from torch.utils.data import Dataset
from torch.utils.data import DataLoader
from torch.utils.data import random_split

from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay

from transformers import (
    SegformerForSemanticSegmentation
)

device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print("Device:", device)

Device: cuda

root_dir =
"/kaggle/input/datasets/rishikeshgopal121/landslide/ReadyToBeUsedDataset"

class SegFormerDataset(Dataset):
    def __init__(self, root_dir):
        self.samples = []

        for region in os.listdir(
            os.path.join(root_dir, "6days")
        ):
            region_path = os.path.join(
                root_dir,
                "6days",
                region
            )

```

```
folders = os.listdir(
    region_path
)

for folder in folders:

    if folder == "Segmentations":
        continue

    coh_path = os.path.join(
        region_path,
        folder,
        "coherence"
    )

    phase_path = os.path.join(
        region_path,
        folder,
        "phase"
    )

    mask_path = os.path.join(
        region_path,
        "Segmentations",
        folder
    )

    if not (
        os.path.exists(coh_path)
        and
        os.path.exists(phase_path)
        and
        os.path.exists(mask_path)
    ):
        continue

    files = sorted(
        os.listdir(coh_path)
    )

    for file in files:

        if file.endswith(".tif"):

            self.samples.append(

                (

                    os.path.join(
                        coh_path,
```

```

        file
    ),
    os.path.join(
        phase_path,
        file
    ),
    os.path.join(
        mask_path,
        file
    )
)

)

print(
    "Total Samples:",
    len(self.samples)
)

def __len__(self):
    return len(
        self.samples
    )

def __getitem__(self, idx):
    coh_file, phase_file, mask_file = self.samples[idx]

    # =====
    # LOAD FILES
    # =====

    coherence = tiff.imread(
        coh_file
    ).astype(np.float32)

    phase = tiff.imread(
        phase_file
    ).astype(np.float32)

    mask = tiff.imread(
        mask_file
    ).astype(np.float32)

    # =====
    # PHASE DECOMPOSITION
    # =====

```

```

cos_phase = np.cos(
    phase
)

sin_phase = np.sin(
    phase
)

# =====
# CREATE INPUT
# =====

image = np.stack(
    [
        coherence,
        cos_phase,
        sin_phase
    ],
    axis=0
)

mask = np.expand_dims(
    mask,
    axis=0
)

# =====
# HORIZONTAL FLIP
# =====

if np.random.rand() > 0.5:
    image = np.flip(
        image,
        axis=2
    ).copy()

    mask = np.flip(
        mask,
        axis=2
    )

```

```

        ).copy()

# =====
# VERTICAL FLIP
# =====

if np.random.rand() > 0.5:

    image = np.flip(
        image,
        axis=1
    ).copy()

    mask = np.flip(
        mask,
        axis=1
    ).copy()

# =====
# ROTATION
# =====

k = np.random.randint(
    0,
    4
)

image = np.rot90(
    image,
    k,
    axes=(1, 2)
).copy()

mask = np.rot90(
    mask,
    k,
    axes=(1, 2)
).copy()

# =====
# TRANSPOSE
# =====

```

```

if np.random.rand() > 0.5:
    image = np.transpose(
        image,
        (0, 2, 1)
    ).copy()
    mask = np.transpose(
        mask,
        (0, 2, 1)
    ).copy()

# =====
# Z-SCORE NORMALIZATION
# =====

image = image.astype(
    np.float32
)

for c in range(
    image.shape[0]
):
    mean = image[c].mean()
    std = image[c].std()
    image[c] = (
        image[c] - mean
    ) / (
        std + 1e-8
    )

mask = mask.astype(
    np.float32
)

return (
    torch.tensor(

```

```

        image,
        dtype=torch.float32
    ),
    torch.tensor(
        mask,
        dtype=torch.float32
    )
)

dataset = SegFormerDataset(
    root_dir
)
Total Samples: 4655
train_size = int(0.70 * len(dataset))
val_size = int(0.15 * len(dataset))
test_size = len(dataset) - train_size - val_size
train_dataset, val_dataset, test_dataset = random_split(
    dataset,
    [
        train_size,
        val_size,
        test_size
    ]
)

print("Train:", len(train_dataset))
print("Validation:", len(val_dataset))
print("Test:", len(test_dataset))

Train: 3258
Validation: 698
Test: 699

train_loader = DataLoader(
    train_dataset,
    batch_size=8,
    shuffle=True
)

```

```

test_loader = DataLoader(
    test_dataset,
    batch_size=8,
    shuffle=False
)
val_loader = DataLoader(
    val_dataset,
    batch_size=8,
    shuffle=False
)
class SegFormerModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.segformer =
SegformerForSemanticSegmentation.from_pretrained(
    "nvidia/segformer-b2-finetuned-ade-512-512",
    num_labels=1,
    ignore_mismatched_sizes=True
)
    def forward(self, x):
        outputs = self.segformer(
            pixel_values=x
        )
        logits = outputs.logits
        logits = F.interpolate(
            logits,
            size=(100, 100),
            mode="bilinear",

```



```

        align_corners=False
    )

    return logits

model = SegFormerModel().to(device)

print(model)

{"model_id": "7b8c3d0649f54b689a90723075baf9d9", "version_major": 2, "version_minor": 0}

```

SegformerForSemanticSegmentation LOAD REPORT from: nvidia/segformer-b2-finetuned-ade-512-512

Key	Status	
-----+-----		
+-----		

decode_head.classifier.weight	MISMATCH	Reinit due to size mismatch ckpt: torch.Size([150, 768, 1, 1]) vs model:torch.Size([1, 768, 1, 1])
decode_head.classifier.bias	MISMATCH	Reinit due to size mismatch ckpt: torch.Size([150]) vs model:torch.Size([1])

Notes:

- MISMATCH :ckpt weights were loaded, but they did not match the original empty weight shapes.

```

SegFormerModel(
  (segformer): SegformerForSemanticSegmentation(
    (segformer): SegformerModel(
      (encoder): SegformerEncoder(
        (patch_embeddings): ModuleList(
          (0): SegformerOverlapPatchEmbeddings(
            (proj): Conv2d(3, 64, kernel_size=(7, 7), stride=(4, 4),
padding=(3, 3))
            (layer_norm): LayerNorm((64,)), eps=1e-05,
elementwise_affine=True)
          )
          (1): SegformerOverlapPatchEmbeddings(
            (proj): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2),
padding=(1, 1))
            (layer_norm): LayerNorm((128,)), eps=1e-05,
elementwise_affine=True)
          )
          (2): SegformerOverlapPatchEmbeddings(
            (proj): Conv2d(128, 320, kernel_size=(3, 3), stride=(2,
2), padding=(1, 1))
            (layer_norm): LayerNorm((320,)), eps=1e-05,

```

```

elementwise_affine=True)
    )
    (3): SegformerOverlapPatchEmbeddings(
      (proj): Conv2d(320, 512, kernel_size=(3, 3), stride=(2,
2), padding=(1, 1))
      (layer_norm): LayerNorm((512,)), eps=1e-05,
elementwise_affine=True)
    )
  )
  (block): ModuleList(
    (0): ModuleList(
      (0): SegformerLayer(
        (layer_norm_1): LayerNorm((64,)), eps=1e-05,
elementwise_affine=True)
        (attention): SegformerAttention(
          (self): SegformerEfficientSelfAttention(
            (query): Linear(in_features=64, out_features=64,
bias=True)
            (key): Linear(in_features=64, out_features=64,
bias=True)
            (value): Linear(in_features=64, out_features=64,
bias=True)
            (dropout): Dropout(p=0.0, inplace=False)
            (sr): Conv2d(64, 64, kernel_size=(8, 8), stride=(8,
8))
            (layer_norm): LayerNorm((64,)), eps=1e-05,
elementwise_affine=True)
          )
          (output): SegformerSelfOutput(
            (dense): Linear(in_features=64, out_features=64,
bias=True)
            (dropout): Dropout(p=0.0, inplace=False)
          )
        )
        (drop_path): Identity()
        (layer_norm_2): LayerNorm((64,)), eps=1e-05,
elementwise_affine=True)
        (mlp): SegformerMixFFN(
          (dense1): Linear(in_features=64, out_features=256,
bias=True)
          (dwconv): SegformerDWConv(
            (dwconv): Conv2d(256, 256, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=256)
          )
          (intermediate_act_fn): GELUActivation()
          (dense2): Linear(in_features=256, out_features=64,
bias=True)
          (dropout): Dropout(p=0.0, inplace=False)
        )
      )
    )
  )
)

```

```

        )
        (1): SegformerLayer(
          (layer_norm_1): LayerNorm((64,), eps=1e-05,
elementwise_affine=True)
          (attention): SegformerAttention(
            (self): SegformerEfficientSelfAttention(
              (query): Linear(in_features=64, out_features=64,
bias=True)
              (key): Linear(in_features=64, out_features=64,
bias=True)
              (value): Linear(in_features=64, out_features=64,
bias=True)
              (dropout): Dropout(p=0.0, inplace=False)
              (sr): Conv2d(64, 64, kernel_size=(8, 8), stride=(8,
8))
              (layer_norm): LayerNorm((64,), eps=1e-05,
elementwise_affine=True)
            )
            (output): SegformerSelfOutput(
              (dense): Linear(in_features=64, out_features=64,
bias=True)
              (dropout): Dropout(p=0.0, inplace=False)
            )
          )
          (drop_path): SegformerDropPath(p=0.0066666666828095913)
          (layer_norm_2): LayerNorm((64,), eps=1e-05,
elementwise_affine=True)
          (mlp): SegformerMixFFN(
            (dense1): Linear(in_features=64, out_features=256,
bias=True)
            (dwconv): SegformerDWConv(
              (dwconv): Conv2d(256, 256, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=256)
            )
            (intermediate_act_fn): GELUActivation()
            (dense2): Linear(in_features=256, out_features=64,
bias=True)
            (dropout): Dropout(p=0.0, inplace=False)
          )
        )
        (2): SegformerLayer(
          (layer_norm_1): LayerNorm((64,), eps=1e-05,
elementwise_affine=True)
          (attention): SegformerAttention(
            (self): SegformerEfficientSelfAttention(
              (query): Linear(in_features=64, out_features=64,
bias=True)
              (key): Linear(in_features=64, out_features=64,
bias=True)

```

```

        (value): Linear(in_features=64, out_features=64,
bias=True)
        (dropout): Dropout(p=0.0, inplace=False)
        (sr): Conv2d(64, 64, kernel_size=(8, 8), stride=(8,
8))
        (layer_norm): LayerNorm((64,), eps=1e-05,
elementwise_affine=True)
    )
    (output): SegformerSelfOutput(
        (dense): Linear(in_features=64, out_features=64,
bias=True)
        (dropout): Dropout(p=0.0, inplace=False)
    )
    (drop_path): SegformerDropPath(p=0.0133333333656191826)
    (layer_norm_2): LayerNorm((64,), eps=1e-05,
elementwise_affine=True)
    (mlp): SegformerMixFFN(
        (dense1): Linear(in_features=64, out_features=256,
bias=True)
        (dwconv): SegformerDWConv(
            (dwconv): Conv2d(256, 256, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=256)
        )
        (intermediate_act_fn): GELUActivation()
        (dense2): Linear(in_features=256, out_features=64,
bias=True)
        (dropout): Dropout(p=0.0, inplace=False)
    )
)
(1): ModuleList(
  (0): SegformerLayer(
    (layer_norm_1): LayerNorm((128,), eps=1e-05,
elementwise_affine=True)
    (attention): SegformerAttention(
      (self): SegformerEfficientSelfAttention(
        (query): Linear(in_features=128, out_features=128,
bias=True)
        (key): Linear(in_features=128, out_features=128,
bias=True)
        (value): Linear(in_features=128, out_features=128,
bias=True)
        (dropout): Dropout(p=0.0, inplace=False)
        (sr): Conv2d(128, 128, kernel_size=(4, 4),
stride=(4, 4))
        (layer_norm): LayerNorm((128,), eps=1e-05,
elementwise_affine=True)
      )
    )
  )
)

```

```

        (output): SegformerSelfOutput(
          (dense): Linear(in_features=128, out_features=128,
bias=True)
          (dropout): Dropout(p=0.0, inplace=False)
        )
      )
      (drop_path): SegformerDropPath(p=0.019999999552965164)
      (layer_norm_2): LayerNorm((128,), eps=1e-05,
elementwise_affine=True)
      (mlp): SegformerMixFFN(
        (dense1): Linear(in_features=128, out_features=512,
bias=True)
        (dwconv): SegformerDWConv(
          (dwconv): Conv2d(512, 512, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=512)
          (intermediate_act_fn): GELUActivation()
          (dense2): Linear(in_features=512, out_features=128,
bias=True)
          (dropout): Dropout(p=0.0, inplace=False)
        )
      )
    (1): SegformerLayer(
      (layer_norm_1): LayerNorm((128,), eps=1e-05,
elementwise_affine=True)
      (attention): SegformerAttention(
        (self): SegformerEfficientSelfAttention(
          (query): Linear(in_features=128, out_features=128,
bias=True)
          (key): Linear(in_features=128, out_features=128,
bias=True)
          (value): Linear(in_features=128, out_features=128,
bias=True)
          (dropout): Dropout(p=0.0, inplace=False)
          (sr): Conv2d(128, 128, kernel_size=(4, 4),
stride=(4, 4))
          (layer_norm): LayerNorm((128,), eps=1e-05,
elementwise_affine=True)
        )
        (output): SegformerSelfOutput(
          (dense): Linear(in_features=128, out_features=128,
bias=True)
          (dropout): Dropout(p=0.0, inplace=False)
        )
      )
      (drop_path): SegformerDropPath(p=0.02666666731238365)
      (layer_norm_2): LayerNorm((128,), eps=1e-05,
elementwise_affine=True)
      (mlp): SegformerMixFFN(

```

```

        (dense1): Linear(in_features=128, out_features=512,
bias=True)
        (dwconv): SegformerDWConv(
          (dwconv): Conv2d(512, 512, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=512)
        )
        (intermediate_act_fn): GELUActivation()
        (dense2): Linear(in_features=512, out_features=128,
bias=True)
        (dropout): Dropout(p=0.0, inplace=False)
      )
    )
    (2): SegformerLayer(
      (layer_norm_1): LayerNorm((128,), eps=1e-05,
elementwise_affine=True)
      (attention): SegformerAttention(
        (self): SegformerEfficientSelfAttention(
          (query): Linear(in_features=128, out_features=128,
bias=True)
          (key): Linear(in_features=128, out_features=128,
bias=True)
          (value): Linear(in_features=128, out_features=128,
bias=True)
          (dropout): Dropout(p=0.0, inplace=False)
          (sr): Conv2d(128, 128, kernel_size=(4, 4),
stride=(4, 4))
          (layer_norm): LayerNorm((128,), eps=1e-05,
elementwise_affine=True)
        )
        (output): SegformerSelfOutput(
          (dense): Linear(in_features=128, out_features=128,
bias=True)
          (dropout): Dropout(p=0.0, inplace=False)
        )
      )
      (drop_path): SegformerDropPath(p=0.03333333507180214)
      (layer_norm_2): LayerNorm((128,), eps=1e-05,
elementwise_affine=True)
      (mlp): SegformerMixFFN(
        (dense1): Linear(in_features=128, out_features=512,
bias=True)
        (dwconv): SegformerDWConv(
          (dwconv): Conv2d(512, 512, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=512)
        )
        (intermediate_act_fn): GELUActivation()
        (dense2): Linear(in_features=512, out_features=128,
bias=True)
        (dropout): Dropout(p=0.0, inplace=False)
      )
    )
  )
)

```

```

        )
    )
    (3): SegformerLayer(
      (layer_norm_1): LayerNorm((128,), eps=1e-05,
elementwise_affine=True)
      (attention): SegformerAttention(
        (self): SegformerEfficientSelfAttention(
          (query): Linear(in_features=128, out_features=128,
bias=True)
          (key): Linear(in_features=128, out_features=128,
bias=True)
          (value): Linear(in_features=128, out_features=128,
bias=True)
          (dropout): Dropout(p=0.0, inplace=False)
          (sr): Conv2d(128, 128, kernel_size=(4, 4),
stride=(4, 4))
          (layer_norm): LayerNorm((128,), eps=1e-05,
elementwise_affine=True)
        )
        (output): SegformerSelfOutput(
          (dense): Linear(in_features=128, out_features=128,
bias=True)
          (dropout): Dropout(p=0.0, inplace=False)
        )
      )
      (drop_path): SegformerDropPath(p=0.039999999910593033)
      (layer_norm_2): LayerNorm((128,), eps=1e-05,
elementwise_affine=True)
      (mlp): SegformerMixFFN(
        (dense1): Linear(in_features=128, out_features=512,
bias=True)
        (dwconv): SegformerDWConv(
          (dwconv): Conv2d(512, 512, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=512)
        )
        (intermediate_act_fn): GELUActivation()
        (dense2): Linear(in_features=512, out_features=128,
bias=True)
        (dropout): Dropout(p=0.0, inplace=False)
      )
    )
  )
(2): ModuleList(
  (0): SegformerLayer(
    (layer_norm_1): LayerNorm((320,), eps=1e-05,
elementwise_affine=True)
    (attention): SegformerAttention(
      (self): SegformerEfficientSelfAttention(
        (query): Linear(in_features=320, out_features=320,

```

```

bias=True)
            (key): Linear(in_features=320, out_features=320,
bias=True)
            (value): Linear(in_features=320, out_features=320,
bias=True)
            (dropout): Dropout(p=0.0, inplace=False)
            (sr): Conv2d(320, 320, kernel_size=(2, 2),
stride=(2, 2))
            (layer_norm): LayerNorm((320,), eps=1e-05,
elementwise_affine=True)
        )
        (output): SegformerSelfOutput(
            (dense): Linear(in_features=320, out_features=320,
bias=True)
            (dropout): Dropout(p=0.0, inplace=False)
        )
    )
    (drop_path): SegformerDropPath(p=0.0466666666865348816)
    (layer_norm_2): LayerNorm((320,), eps=1e-05,
elementwise_affine=True)
    (mlp): SegformerMixFFN(
        (dense1): Linear(in_features=320, out_features=1280,
bias=True)
        (dwconv): SegformerDWConv(
            (dwconv): Conv2d(1280, 1280, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=1280)
        )
        (intermediate_act_fn): GELUActivation()
        (dense2): Linear(in_features=1280, out_features=320,
bias=True)
        (dropout): Dropout(p=0.0, inplace=False)
    )
)
(1): SegformerLayer(
    (layer_norm_1): LayerNorm((320,), eps=1e-05,
elementwise_affine=True)
    (attention): SegformerAttention(
        (self): SegformerEfficientSelfAttention(
            (query): Linear(in_features=320, out_features=320,
bias=True)
            (key): Linear(in_features=320, out_features=320,
bias=True)
            (value): Linear(in_features=320, out_features=320,
bias=True)
            (dropout): Dropout(p=0.0, inplace=False)
            (sr): Conv2d(320, 320, kernel_size=(2, 2),
stride=(2, 2))
            (layer_norm): LayerNorm((320,), eps=1e-05,
elementwise_affine=True)

```



```

        )
        (output): SegformerSelfOutput(
          (dense): Linear(in_features=320, out_features=320,
bias=True)
          (dropout): Dropout(p=0.0, inplace=False)
        )
      )
      (drop_path): SegformerDropPath(p=0.05333333346247673)
      (layer_norm_2): LayerNorm((320,), eps=1e-05,
elementwise_affine=True)
      (mlp): SegformerMixFFN(
        (dense1): Linear(in_features=320, out_features=1280,
bias=True)
        (dwconv): SegformerDWConv(
          (dwconv): Conv2d(1280, 1280, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=1280)
        )
        (intermediate_act_fn): GELUActivation()
        (dense2): Linear(in_features=1280, out_features=320,
bias=True)
        (dropout): Dropout(p=0.0, inplace=False)
      )
    )
    (2): SegformerLayer(
      (layer_norm_1): LayerNorm((320,), eps=1e-05,
elementwise_affine=True)
      (attention): SegformerAttention(
        (self): SegformerEfficientSelfAttention(
          (query): Linear(in_features=320, out_features=320,
bias=True)
          (key): Linear(in_features=320, out_features=320,
bias=True)
          (value): Linear(in_features=320, out_features=320,
bias=True)
          (dropout): Dropout(p=0.0, inplace=False)
        )
        (sr): Conv2d(320, 320, kernel_size=(2, 2),
stride=(2, 2))
        (layer_norm): LayerNorm((320,), eps=1e-05,
elementwise_affine=True)
      )
      (output): SegformerSelfOutput(
        (dense): Linear(in_features=320, out_features=320,
bias=True)
        (dropout): Dropout(p=0.0, inplace=False)
      )
    )
    (drop_path): SegformerDropPath(p=0.060000000238418579)
    (layer_norm_2): LayerNorm((320,), eps=1e-05,
elementwise_affine=True)

```

```

        (mlp): SegformerMixFFN(
          (dense1): Linear(in_features=320, out_features=1280,
bias=True)
          (dwconv): SegformerDWConv(
            (dwconv): Conv2d(1280, 1280, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=1280)
            )
          (intermediate_act_fn): GELUActivation()
          (dense2): Linear(in_features=1280, out_features=320,
bias=True)
          (dropout): Dropout(p=0.0, inplace=False)
        )
      )
    (3): SegformerLayer(
      (layer_norm_1): LayerNorm((320,), eps=1e-05,
elementwise_affine=True)
      (attention): SegformerAttention(
        (self): SegformerEfficientSelfAttention(
          (query): Linear(in_features=320, out_features=320,
bias=True)
          (key): Linear(in_features=320, out_features=320,
bias=True)
          (value): Linear(in_features=320, out_features=320,
bias=True)
          (dropout): Dropout(p=0.0, inplace=False)
          (sr): Conv2d(320, 320, kernel_size=(2, 2),
stride=(2, 2))
          (layer_norm): LayerNorm((320,), eps=1e-05,
elementwise_affine=True)
        )
        (output): SegformerSelfOutput(
          (dense): Linear(in_features=320, out_features=320,
bias=True)
          (dropout): Dropout(p=0.0, inplace=False)
        )
      )
      (drop_path): SegformerDropPath(p=0.06666667014360428)
      (layer_norm_2): LayerNorm((320,), eps=1e-05,
elementwise_affine=True)
      (mlp): SegformerMixFFN(
        (dense1): Linear(in_features=320, out_features=1280,
bias=True)
        (dwconv): SegformerDWConv(
          (dwconv): Conv2d(1280, 1280, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=1280)
          )
        (intermediate_act_fn): GELUActivation()
        (dense2): Linear(in_features=1280, out_features=320,
bias=True)

```

```

        (dropout): Dropout(p=0.0, inplace=False)
    )
)
(4): SegformerLayer(
  (layer_norm_1): LayerNorm((320,), eps=1e-05,
elementwise_affine=True)
  (attention): SegformerAttention(
    (self): SegformerEfficientSelfAttention(
      (query): Linear(in_features=320, out_features=320,
bias=True)
      (key): Linear(in_features=320, out_features=320,
bias=True)
      (value): Linear(in_features=320, out_features=320,
bias=True)
      (dropout): Dropout(p=0.0, inplace=False)
      (sr): Conv2d(320, 320, kernel_size=(2, 2),
stride=(2, 2))
      (layer_norm): LayerNorm((320,), eps=1e-05,
elementwise_affine=True)
    )
    (output): SegformerSelfOutput(
      (dense): Linear(in_features=320, out_features=320,
bias=True)
      (dropout): Dropout(p=0.0, inplace=False)
    )
  )
  (drop_path): SegformerDropPath(p=0.07333333790302277)
  (layer_norm_2): LayerNorm((320,), eps=1e-05,
elementwise_affine=True)
  (mlp): SegformerMixFFN(
    (dense1): Linear(in_features=320, out_features=1280,
bias=True)
    (dwconv): SegformerDWConv(
      (dwconv): Conv2d(1280, 1280, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=1280)
    )
    (intermediate_act_fn): GELUActivation()
    (dense2): Linear(in_features=1280, out_features=320,
bias=True)
    (dropout): Dropout(p=0.0, inplace=False)
  )
)
(5): SegformerLayer(
  (layer_norm_1): LayerNorm((320,), eps=1e-05,
elementwise_affine=True)
  (attention): SegformerAttention(
    (self): SegformerEfficientSelfAttention(
      (query): Linear(in_features=320, out_features=320,
bias=True)
      (key): Linear(in_features=320, out_features=320,

```

```

bias=True)
        (value): Linear(in_features=320, out_features=320,
bias=True)
        (dropout): Dropout(p=0.0, inplace=False)
        (sr): Conv2d(320, 320, kernel_size=(2, 2),
stride=(2, 2))
        (layer_norm): LayerNorm((320,), eps=1e-05,
elementwise_affine=True)
    )
    (output): SegformerSelfOutput(
        (dense): Linear(in_features=320, out_features=320,
bias=True)
        (dropout): Dropout(p=0.0, inplace=False)
    )
    (drop_path): SegformerDropPath(p=0.07999999821186066)
    (layer_norm_2): LayerNorm((320,), eps=1e-05,
elementwise_affine=True)
    (mlp): SegformerMixFFN(
        (dense1): Linear(in_features=320, out_features=1280,
bias=True)
        (dwconv): SegformerDWConv(
            (dwconv): Conv2d(1280, 1280, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=1280)
        )
        (intermediate_act_fn): GELUActivation()
        (dense2): Linear(in_features=1280, out_features=320,
bias=True)
        (dropout): Dropout(p=0.0, inplace=False)
    )
)
(3): ModuleList(
  (0): SegformerLayer(
    (layer_norm_1): LayerNorm((512,), eps=1e-05,
elementwise_affine=True)
    (attention): SegformerAttention(
      (self): SegformerEfficientSelfAttention(
        (query): Linear(in_features=512, out_features=512,
bias=True)
        (key): Linear(in_features=512, out_features=512,
bias=True)
        (value): Linear(in_features=512, out_features=512,
bias=True)
        (dropout): Dropout(p=0.0, inplace=False)
      )
      (output): SegformerSelfOutput(
        (dense): Linear(in_features=512, out_features=512,
bias=True)

```

```

        (dropout): Dropout(p=0.0, inplace=False)
    )
    (drop_path): SegformerDropPath(p=0.08666666597127914)
    (layer_norm_2): LayerNorm((512,), eps=1e-05,
elementwise_affine=True)
    (mlp): SegformerMixFFN(
        (dense1): Linear(in_features=512, out_features=2048,
bias=True)
        (dwconv): SegformerDWConv(
            (dwconv): Conv2d(2048, 2048, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=2048)
        )
        (intermediate_act_fn): GELUActivation()
        (dense2): Linear(in_features=2048, out_features=512,
bias=True)
        (dropout): Dropout(p=0.0, inplace=False)
    )
    )
    (1): SegformerLayer(
        (layer_norm_1): LayerNorm((512,), eps=1e-05,
elementwise_affine=True)
        (attention): SegformerAttention(
            (self): SegformerEfficientSelfAttention(
                (query): Linear(in_features=512, out_features=512,
bias=True)
                (key): Linear(in_features=512, out_features=512,
bias=True)
                (value): Linear(in_features=512, out_features=512,
bias=True)
                (dropout): Dropout(p=0.0, inplace=False)
            )
            (output): SegformerSelfOutput(
                (dense): Linear(in_features=512, out_features=512,
bias=True)
                (dropout): Dropout(p=0.0, inplace=False)
            )
        )
        (drop_path): SegformerDropPath(p=0.09333333373069763)
        (layer_norm_2): LayerNorm((512,), eps=1e-05,
elementwise_affine=True)
        (mlp): SegformerMixFFN(
            (dense1): Linear(in_features=512, out_features=2048,
bias=True)
            (dwconv): SegformerDWConv(
                (dwconv): Conv2d(2048, 2048, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=2048)
            )
            (intermediate_act_fn): GELUActivation()

```

```

        (dense2): Linear(in_features=2048, out_features=512,
bias=True)
        (dropout): Dropout(p=0.0, inplace=False)
    )
    )
    (2): SegformerLayer(
        (layer_norm_1): LayerNorm((512,), eps=1e-05,
elementwise_affine=True)
        (attention): SegformerAttention(
            (self): SegformerEfficientSelfAttention(
                (query): Linear(in_features=512, out_features=512,
bias=True)
                (key): Linear(in_features=512, out_features=512,
bias=True)
                (value): Linear(in_features=512, out_features=512,
bias=True)
                (dropout): Dropout(p=0.0, inplace=False)
            )
            (output): SegformerSelfOutput(
                (dense): Linear(in_features=512, out_features=512,
bias=True)
                (dropout): Dropout(p=0.0, inplace=False)
            )
        )
        (drop_path): SegformerDropPath(p=0.10000000149011612)
        (layer_norm_2): LayerNorm((512,), eps=1e-05,
elementwise_affine=True)
        (mlp): SegformerMixFFN(
            (dense1): Linear(in_features=512, out_features=2048,
bias=True)
            (dwconv): SegformerDWConv(
                (dwconv): Conv2d(2048, 2048, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=2048)
            )
            (intermediate_act_fn): GELUActivation()
            (dense2): Linear(in_features=2048, out_features=512,
bias=True)
            (dropout): Dropout(p=0.0, inplace=False)
        )
    )
)
)
(layer_norm): ModuleList(
  (0): LayerNorm((64,), eps=1e-05, elementwise_affine=True)
  (1): LayerNorm((128,), eps=1e-05, elementwise_affine=True)
  (2): LayerNorm((320,), eps=1e-05, elementwise_affine=True)
  (3): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
)
)

```

```

    )
    (decode_head): SegformerDecodeHead(
      (linear_c): ModuleList(
        (0): SegformerMLP(
          (proj): Linear(in_features=64, out_features=768, bias=True)
        )
        (1): SegformerMLP(
          (proj): Linear(in_features=128, out_features=768, bias=True)
        )
        (2): SegformerMLP(
          (proj): Linear(in_features=320, out_features=768, bias=True)
        )
        (3): SegformerMLP(
          (proj): Linear(in_features=512, out_features=768, bias=True)
        )
      )
      (linear_fuse): Conv2d(3072, 768, kernel_size=(1, 1), stride=(1,
1), bias=False)
      (batch_norm): BatchNorm2d(768, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True)
      (activation): ReLU()
      (dropout): Dropout(p=0.1, inplace=False)
      (classifier): Conv2d(768, 1, kernel_size=(1, 1), stride=(1, 1))
    )
  )
)

sample_batch = next(iter(train_loader))

images, masks = sample_batch

images = images.to(device)

with torch.no_grad():

    outputs = model(images)

print("Input Shape :", images.shape)

print("Output Shape:", outputs.shape)

Input Shape : torch.Size([8, 3, 100, 100])
Output Shape: torch.Size([8, 1, 100, 100])


# =====
#  loss function :
#1. DICE LOSS
# =====

```

```

def dice_loss(pred, target):
    pred = torch.sigmoid(pred)
    pred = pred.view(-1)
    target = target.view(-1)
    intersection = (
        pred * target
    ).sum()
    dice = (
        2 * intersection + 1e-8
    ) / (
        pred.sum()
        + target.sum()
        + 1e-8
    )
    return 1 - dice

# =====
# FOCAL LOSS
# =====

def focal_loss(
    pred,
    target,
    alpha=0.8,
    gamma=2
):
    bce = F.binary_cross_entropy_with_logits(
        pred,
        target,

```



```

        reduction="none"
    )
    pt = torch.exp(-bce)
    focal = (
        alpha
        *
        ((1 - pt) ** gamma)
        *
        bce
    )
    return focal.mean()

# =====
# TVERSKY LOSS
# =====

def tversky_loss(
    pred,
    target,
    alpha=0.3,
    beta=0.7
):
    pred = torch.sigmoid(pred)
    pred = pred.view(-1)
    target = target.view(-1)
    TP = (
        pred * target
    ).sum()

```

```

    FP = (
        pred * (1 - target)
    ).sum()
    FN = (
        (1 - pred) * target
    ).sum()
    tversky = (
        TP + 1e-8
    ) / (
        TP
        + alpha * FP
        + beta * FN
        + 1e-8
    )
    return 1 - tversky

# =====
# HYBRID LOSS
# =====

def hybrid_loss(
    pred,
    target
):
    d = dice_loss(
        pred,
        target
    )
    f = focal_loss(

```

```

        pred,
        target
    )
    t = tversky_loss(
        pred,
        target
    )
    total_loss = (
        0.3 * d
        +
        0.2 * f
        +
        0.5 * t
    )
    return total_loss
optimizer = optim.AdamW(
    model.parameters(),
    lr=0.0001,
    weight_decay=1e-4
)
scheduler = optim.lr_scheduler.ReduceLR0nPlateau(
    optimizer,
    mode='min',
    factor=0.5,
    patience=5
)

```

```

def calculate_all_metrics(pred, target):
    pred = (pred > 0.5).float()
    TP = (pred * target).sum()
    TN = ((1-pred) * (1-target)).sum()
    FP = (pred * (1-target)).sum()
    FN = ((1-pred) * target).sum()
    accuracy = (
        TP + TN
    ) / (
        TP + TN + FP + FN + 1e-8
    )
    precision = TP / (
        TP + FP + 1e-8
    )
    recall = TP / (
        TP + FN + 1e-8
    )
    f1 = (
        2 * TP
    ) / (
        2 * TP + FP + FN + 1e-8
    )
    dice = (
        2 * TP
    ) / (
        2 * TP + FP + FN + 1e-8
    )

```

```

    )
    iou = TP / (
        TP + FP + FN + 1e-8
    )
    return (
        accuracy.item(),
        precision.item(),
        recall.item(),
        f1.item(),
        dice.item(),
        iou.item()
    )
train_losses = []
train_acc = []
train_prec = []
train_rec = []
val_losses = []
val_acc = []
val_prec = []
val_rec = []
val_f1 = []
val_dice = []
val_iou = []
epochs = 150
best_loss = float('inf')
patience = 15
counter = 0
for epoch in range(epochs):
    # =====

```

```

# TRAINING
# =====

model.train()

total_loss = 0

correct = 0
total = 0

TP = 0
FP = 0
FN = 0

print(f"\nEpoch {epoch+1}/{epochs}")

for i, (images, masks) in enumerate(train_loader):
    images = images.to(device)
    masks = masks.to(device)

    optimizer.zero_grad()

    outputs = model(images)

    loss = hybrid_loss(
        outputs,
        masks
    )

    loss.backward()

    torch.nn.utils.clip_grad_norm_(
        model.parameters(),
        1.0
    )

    optimizer.step()

    total_loss += loss.item()

    pred = torch.sigmoid(outputs)

    pred_bin = (
        pred > 0.5
    ).float()

    correct += (
        pred_bin == masks
    ).sum().item()

```

```

        total += masks.numel()

        TP += (
            pred_bin * masks
        ).sum().item()

        FP += (
            pred_bin * (1 - masks)
        ).sum().item()

        FN += (
            (1 - pred_bin) * masks
        ).sum().item()

        progress = (
            (i + 1)
            /
            len(train_loader)
        ) * 100

        print(
            f"\rProgress: {progress:.2f}% | Loss: {loss.item():.4f}",
            end=""
        )

# =====
# TRAIN METRICS
# =====

acc = correct / total

precision = TP / (
    TP + FP + 1e-8
)

recall = TP / (
    TP + FN + 1e-8
)

train_losses.append(
    total_loss / len(train_loader)
)

train_acc.append(acc)

train_prec.append(
    precision
)

train_rec.append(

```

```

        recall
    )

# =====
# VALIDATION
# =====

model.eval()

v_loss = 0

v_acc = 0
v_prec = 0
v_rec = 0

v_f1 = 0
v_dice = 0
v_iou = 0

v_count = 0

with torch.no_grad():
    for images, masks in val_loader:
        images = images.to(device)
        masks = masks.to(device)
        outputs = model(images)

        loss = hybrid_loss(
            outputs,
            masks
        )

        v_loss += loss.item()

        pred = torch.sigmoid(outputs)

        a, p, r, f1, d, i = calculate_all_metrics(
            pred,
            masks
        )

        v_acc += a
        v_prec += p
        v_rec += r

        v_f1 += f1
        v_dice += d

```



```

        v_iou += i

        v_count += 1

    val_losses.append(
        v_loss / v_count
    )

    val_acc.append(
        v_acc / v_count
    )

    val_prec.append(
        v_prec / v_count
    )

    val_rec.append(
        v_rec / v_count
    )

    val_f1.append(
        v_f1 / v_count
    )

    val_dice.append(
        v_dice / v_count
    )

    val_iou.append(
        v_iou / v_count
    )

# =====
# PRINT METRICS
# =====

print(f"\nTrain Loss      : {train_losses[-1]:.4f}")
print(f"Train Accuracy   : {train_acc[-1]:.4f}")
print(f"Train Precision    : {train_prec[-1]:.4f}")
print(f"Train Recall       : {train_rec[-1]:.4f}")

print()

print(f"Val Loss           : {val_losses[-1]:.4f}")
print(f"Val Accuracy       : {val_acc[-1]:.4f}")
print(f"Val Precision      : {val_prec[-1]:.4f}")
print(f"Val Recall         : {val_rec[-1]:.4f}")
print(f"Val F1             : {val_f1[-1]:.4f}")
print(f"Val Dice           : {val_dice[-1]:.4f}")
print(f"Val IoU            : {val_iou[-1]:.4f}")

```

```

# =====
# LR SCHEDULER
# =====

scheduler.step(
    val_losses[-1]
)

# =====
# MODEL SAVING + EARLY STOPPING
# =====

if val_losses[-1] < best_loss:
    best_loss = val_losses[-1]
    counter = 0
    torch.save(
        model.state_dict(),
        "segformer_best.pth"
    )
    print("\nModel Saved!")
else:
    counter += 1
    print(
        f"\nNo Improvement: {counter}/{patience}"
    )
    if counter >= patience:
        print(
            f"\nEarly Stopping Triggered after Epoch {epoch+1}"
        )
        break

```

```

Epoch 1/150
Progress: 100.00% | Loss: 0.8197
Train Loss      : 0.5058
Train Accuracy  : 0.9455
Train Precision : 0.3318

```

Train Recall : 0.6926

Val Loss : 0.3502

Val Accuracy : 0.9774

Val Precision : 0.6556

Val Recall : 0.6257

Val F1 : 0.6191

Val Dice : 0.6191

Val IoU : 0.4664

Model Saved!

Epoch 2/150

Progress: 100.00% | Loss: 0.3853

Train Loss : 0.1943

Train Accuracy : 0.9865

Train Precision : 0.7703

Train Recall : 0.8226

Val Loss : 0.2303

Val Accuracy : 0.9834

Val Precision : 0.7192

Val Recall : 0.7604

Val F1 : 0.7288

Val Dice : 0.7288

Val IoU : 0.5862

Model Saved!

Epoch 13/150

Progress: 100.00% | Loss: 0.1201

Train Loss : 0.1887

Train Accuracy : 0.9867

Train Precision : 0.7698

Train Recall : 0.8367

Val Loss : 0.2300

Val Accuracy : 0.9840

Val Precision : 0.7559

Val Recall : 0.7413

Val F1 : 0.7349

Val Dice : 0.7349

Val IoU : 0.5978

Model Saved!

Epoch 14/150

Progress: 100.00% | Loss: 0.2050

Train Loss : 0.1813

Train Accuracy : 0.9871

Train Precision : 0.7782
Train Recall : 0.8345

Val Loss : 0.2232
Val Accuracy : 0.9823
Val Precision : 0.6996
Val Recall : 0.7942
Val F1 : 0.7330
Val Dice : 0.7330
Val IoU : 0.5887

Model Saved!

Epoch 15/150

Progress: 100.00% | Loss: 0.0605

Train Loss : 0.1792
Train Accuracy : 0.9876
Train Precision : 0.7868
Train Recall : 0.8428

Val Loss : 0.2286
Val Accuracy : 0.9835
Val Precision : 0.7229
Val Recall : 0.7620
Val F1 : 0.7303
Val Dice : 0.7303
Val IoU : 0.5932

No Improvement: 1/15

Epoch 16/150

Progress: 100.00% | Loss: 0.3407

Train Loss : 0.1687
Train Accuracy : 0.9878
Train Precision : 0.7870
Train Recall : 0.8503

Val Loss : 0.2200
Val Accuracy : 0.9813
Val Precision : 0.6735
Val Recall : 0.8296
Val F1 : 0.7302
Val Dice : 0.7302
Val IoU : 0.5906

Model Saved!

Epoch 17/150

Progress: 100.00% | Loss: 0.0975

Train Loss : 0.1670

Train Accuracy : 0.9883
Train Precision : 0.7957
Train Recall : 0.8533

Val Loss : 0.2256
Val Accuracy : 0.9838
Val Precision : 0.7431
Val Recall : 0.7566
Val F1 : 0.7385
Val Dice : 0.7385
Val IoU : 0.6026

No Improvement: 1/15

Epoch 18/150

Progress: 100.00% | Loss: 0.2229

Train Loss : 0.1595
Train Accuracy : 0.9886
Train Precision : 0.8006
Train Recall : 0.8593

Val Loss : 0.2220
Val Accuracy : 0.9823
Val Precision : 0.6966
Val Recall : 0.8056
Val F1 : 0.7326
Val Dice : 0.7326
Val IoU : 0.5918

No Improvement: 2/15

Epoch 19/150

Progress: 100.00% | Loss: 0.1168

Train Loss : 0.1627
Train Accuracy : 0.9887
Train Precision : 0.8001
Train Recall : 0.8645

Val Loss : 0.2092
Val Accuracy : 0.9840
Val Precision : 0.7328
Val Recall : 0.7992
Val F1 : 0.7529
Val Dice : 0.7529
Val IoU : 0.6166

Model Saved!

Epoch 20/150

Progress: 100.00% | Loss: 0.1129

Train Loss : 0.1542
Train Accuracy : 0.9890
Train Precision : 0.8038
Train Recall : 0.8675

Val Loss : 0.2098
Val Accuracy : 0.9831
Val Precision : 0.7036
Val Recall : 0.8248
Val F1 : 0.7472
Val Dice : 0.7472
Val IoU : 0.6089

No Improvement: 1/15

Epoch 21/150

Progress: 100.00% | Loss: 0.1503

Train Loss : 0.1527
Train Accuracy : 0.9894
Train Precision : 0.8106
Train Recall : 0.8716

Val Loss : 0.2017
Val Accuracy : 0.9858
Val Precision : 0.7936
Val Recall : 0.7692
Val F1 : 0.7722
Val Dice : 0.7722
Val IoU : 0.6389

Model Saved!

Epoch 22/150

Progress: 100.00% | Loss: 0.0658

Train Loss : 0.1467
Train Accuracy : 0.9896
Train Precision : 0.8161
Train Recall : 0.8736

Val Loss : 0.2092
Val Accuracy : 0.9849
Val Precision : 0.7637
Val Recall : 0.7759
Val F1 : 0.7594
Val Dice : 0.7594
Val IoU : 0.6244

No Improvement: 1/15

Epoch 23/150

Progress: 100.00% | Loss: 0.0741

Train Loss : 0.1433
Train Accuracy : 0.9898
Train Precision : 0.8169
Train Recall : 0.8770

Val Loss : 0.2004
Val Accuracy : 0.9846
Val Precision : 0.7259
Val Recall : 0.8229
Val F1 : 0.7603
Val Dice : 0.7603
Val IoU : 0.6260

Model Saved!

Epoch 24/150

Progress: 100.00% | Loss: 0.1359

Train Loss : 0.1407
Train Accuracy : 0.9899
Train Precision : 0.8177
Train Recall : 0.8805

Val Loss : 0.2053
Val Accuracy : 0.9840
Val Precision : 0.7196
Val Recall : 0.8200
Val F1 : 0.7538
Val Dice : 0.7538
Val IoU : 0.6187

No Improvement: 1/15

Epoch 25/150

Progress: 100.00% | Loss: 0.1347

Train Loss : 0.1423
Train Accuracy : 0.9901
Train Precision : 0.8201
Train Recall : 0.8837

Val Loss : 0.2058
Val Accuracy : 0.9849
Val Precision : 0.7638
Val Recall : 0.7834
Val F1 : 0.7645
Val Dice : 0.7645
Val IoU : 0.6288

No Improvement: 2/15

Epoch 26/150
Progress: 100.00% | Loss: 0.1344
Train Loss : 0.1365
Train Accuracy : 0.9903
Train Precision : 0.8250
Train Recall : 0.8851

Val Loss : 0.1996
Val Accuracy : 0.9852
Val Precision : 0.7595
Val Recall : 0.7935
Val F1 : 0.7680
Val Dice : 0.7680
Val IoU : 0.6372

Model Saved!

Epoch 27/150
Progress: 100.00% | Loss: 0.0899
Train Loss : 0.1325
Train Accuracy : 0.9904
Train Precision : 0.8266
Train Recall : 0.8882

Val Loss : 0.1975
Val Accuracy : 0.9860
Val Precision : 0.7777
Val Recall : 0.7880
Val F1 : 0.7740
Val Dice : 0.7740
Val IoU : 0.6417

Model Saved!

Epoch 28/150
Progress: 100.00% | Loss: 0.1162
Train Loss : 0.1298
Train Accuracy : 0.9907
Train Precision : 0.8318
Train Recall : 0.8889

Val Loss : 0.1979
Val Accuracy : 0.9864
Val Precision : 0.8043
Val Recall : 0.7687
Val F1 : 0.7774
Val Dice : 0.7774
Val IoU : 0.6439

No Improvement: 1/15

Epoch 29/150
Progress: 100.00% | Loss: 0.0939
Train Loss : 0.1279
Train Accuracy : 0.9910
Train Precision : 0.8368
Train Recall : 0.8926

Val Loss : 0.1968
Val Accuracy : 0.9862
Val Precision : 0.7729
Val Recall : 0.7881
Val F1 : 0.7727
Val Dice : 0.7727
Val IoU : 0.6421

Model Saved!

Epoch 30/150
Progress: 100.00% | Loss: 0.0558
Train Loss : 0.1238
Train Accuracy : 0.9910
Train Precision : 0.8352
Train Recall : 0.8969

Val Loss : 0.1948
Val Accuracy : 0.9860
Val Precision : 0.7760
Val Recall : 0.7964
Val F1 : 0.7760
Val Dice : 0.7760
Val IoU : 0.6427

Model Saved!

Epoch 31/150
Progress: 100.00% | Loss: 0.1354
Train Loss : 0.1227
Train Accuracy : 0.9911
Train Precision : 0.8368
Train Recall : 0.8985

Val Loss : 0.1931
Val Accuracy : 0.9860
Val Precision : 0.7643
Val Recall : 0.8029
Val F1 : 0.7742
Val Dice : 0.7742
Val IoU : 0.6451

Model Saved!

Epoch 32/150

Progress: 100.00% | Loss: 0.8000

Train Loss : 0.1192

Train Accuracy : 0.9914

Train Precision : 0.8400

Train Recall : 0.9022

Val Loss : 0.1920

Val Accuracy : 0.9855

Val Precision : 0.7377

Val Recall : 0.8234

Val F1 : 0.7713

Val Dice : 0.7713

Val IoU : 0.6386

Model Saved!

Epoch 33/150

Progress: 100.00% | Loss: 0.0998

Train Loss : 0.1172

Train Accuracy : 0.9915

Train Precision : 0.8440

Train Recall : 0.9027

Val Loss : 0.2010

Val Accuracy : 0.9852

Val Precision : 0.7526

Val Recall : 0.8008

Val F1 : 0.7655

Val Dice : 0.7655

Val IoU : 0.6334

No Improvement: 1/15

Epoch 34/150

Progress: 100.00% | Loss: 0.0848

Train Loss : 0.1179

Train Accuracy : 0.9914

Train Precision : 0.8419

Train Recall : 0.9014

Val Loss : 0.2012

Val Accuracy : 0.9845

Val Precision : 0.7247

Val Recall : 0.8275

Val F1 : 0.7594

Val Dice : 0.7594

Val IoU : 0.6258

No Improvement: 2/15

Epoch 35/150

Progress: 100.00% | Loss: 0.0420

Train Loss : 0.1118

Train Accuracy : 0.9919

Train Precision : 0.8488

Train Recall : 0.9074

Val Loss : 0.1994

Val Accuracy : 0.9857

Val Precision : 0.7544

Val Recall : 0.8016

Val F1 : 0.7677

Val Dice : 0.7677

Val IoU : 0.6370

No Improvement: 3/15

Epoch 36/150

Progress: 100.00% | Loss: 0.8000

Train Loss : 0.1114

Train Accuracy : 0.9919

Train Precision : 0.8483

Train Recall : 0.9090

Val Loss : 0.1908

Val Accuracy : 0.9864

Val Precision : 0.7631

Val Recall : 0.8056

Val F1 : 0.7780

Val Dice : 0.7780

Val IoU : 0.6491

Model Saved!

Epoch 37/150

Progress: 100.00% | Loss: 0.2001

Train Loss : 0.1124

Train Accuracy : 0.9920

Train Precision : 0.8494

Train Recall : 0.9114

Val Loss : 0.1958

Val Accuracy : 0.9855

Val Precision : 0.7386

Val Recall : 0.8196

Val F1 : 0.7674

Val Dice : 0.7674

Val IoU : 0.6359

No Improvement: 1/15

Epoch 38/150

Progress: 100.00% | Loss: 0.0630

Train Loss : 0.1098

Train Accuracy : 0.9921

Train Precision : 0.8514

Train Recall : 0.9120

Val Loss : 0.1961

Val Accuracy : 0.9864

Val Precision : 0.7857

Val Recall : 0.7864

Val F1 : 0.7781

Val Dice : 0.7781

Val IoU : 0.6482

No Improvement: 2/15

Epoch 39/150

Progress: 100.00% | Loss: 0.8000

Train Loss : 0.1106

Train Accuracy : 0.9922

Train Precision : 0.8543

Train Recall : 0.9136

Val Loss : 0.1802

Val Accuracy : 0.9868

Val Precision : 0.7582

Val Recall : 0.8318

Val F1 : 0.7867

Val Dice : 0.7867

Val IoU : 0.6593

Model Saved!

Epoch 40/150

Progress: 100.00% | Loss: 0.8002

Train Loss : 0.1085

Train Accuracy : 0.9922

Train Precision : 0.8542

Train Recall : 0.9139

Val Loss : 0.1798

Val Accuracy : 0.9869

Val Precision : 0.7677

Val Recall : 0.8320

Val F1 : 0.7893

Val Dice : 0.7893

Val IoU : 0.6614

Model Saved!

Epoch 41/150

Progress: 100.00% | Loss: 0.1746

Train Loss : 0.1064

Train Accuracy : 0.9924

Train Precision : 0.8565

Train Recall : 0.9159

Val Loss : 0.1788

Val Accuracy : 0.9868

Val Precision : 0.7626

Val Recall : 0.8329

Val F1 : 0.7899

Val Dice : 0.7899

Val IoU : 0.6636

Model Saved!

Epoch 42/150

Progress: 100.00% | Loss: 0.8000

Train Loss : 0.1045

Train Accuracy : 0.9925

Train Precision : 0.8583

Train Recall : 0.9177

Val Loss : 0.2001

Val Accuracy : 0.9862

Val Precision : 0.7626

Val Recall : 0.7965

Val F1 : 0.7699

Val Dice : 0.7699

Val IoU : 0.6385

No Improvement: 1/15

Epoch 43/150

Progress: 100.00% | Loss: 0.1648

Train Loss : 0.1020

Train Accuracy : 0.9925

Train Precision : 0.8597

Train Recall : 0.9164

Val Loss : 0.1831

Val Accuracy : 0.9869

Val Precision : 0.7915

Val Recall : 0.8036

Val F1 : 0.7910

Val Dice : 0.7910
Val IoU : 0.6617

No Improvement: 2/15

Epoch 44/150

Progress: 100.00% | Loss: 0.0641

Train Loss : 0.0993
Train Accuracy : 0.9927
Train Precision : 0.8624
Train Recall : 0.9196

Val Loss : 0.1864
Val Accuracy : 0.9864
Val Precision : 0.7677
Val Recall : 0.8157
Val F1 : 0.7842
Val Dice : 0.7842
Val IoU : 0.6571

No Improvement: 3/15

Epoch 45/150

Progress: 100.00% | Loss: 0.0961

Train Loss : 0.0952
Train Accuracy : 0.9929
Train Precision : 0.8640
Train Recall : 0.9237

Val Loss : 0.1906
Val Accuracy : 0.9865
Val Precision : 0.7755
Val Recall : 0.8028
Val F1 : 0.7813
Val Dice : 0.7813
Val IoU : 0.6527

No Improvement: 4/15

Epoch 46/150

Progress: 100.00% | Loss: 0.0421

Train Loss : 0.0986
Train Accuracy : 0.9928
Train Precision : 0.8631
Train Recall : 0.9202

Val Loss : 0.1755
Val Accuracy : 0.9868
Val Precision : 0.7735
Val Recall : 0.8335

Val F1 : 0.7953
Val Dice : 0.7953
Val IoU : 0.6728

Model Saved!

Epoch 47/150

Progress: 100.00% | Loss: 0.1247

Train Loss : 0.0961
Train Accuracy : 0.9928
Train Precision : 0.8620
Train Recall : 0.9245

Val Loss : 0.1834
Val Accuracy : 0.9872
Val Precision : 0.7968
Val Recall : 0.8039
Val F1 : 0.7930
Val Dice : 0.7930
Val IoU : 0.6652

No Improvement: 1/15

Epoch 48/150

Progress: 100.00% | Loss: 0.0666

Train Loss : 0.0946
Train Accuracy : 0.9931
Train Precision : 0.8677
Train Recall : 0.9251

Val Loss : 0.1890
Val Accuracy : 0.9870
Val Precision : 0.7959
Val Recall : 0.7938
Val F1 : 0.7876
Val Dice : 0.7876
Val IoU : 0.6634

No Improvement: 2/15

Epoch 49/150

Progress: 100.00% | Loss: 0.1297

Train Loss : 0.0936
Train Accuracy : 0.9931
Train Precision : 0.8682
Train Recall : 0.9232

Val Loss : 0.1874
Val Accuracy : 0.9862
Val Precision : 0.7606

Val Recall : 0.8187
Val F1 : 0.7811
Val Dice : 0.7811
Val IoU : 0.6511

No Improvement: 3/15

Epoch 50/150

Progress: 100.00% | Loss: 0.1471

Train Loss : 0.0925
Train Accuracy : 0.9933
Train Precision : 0.8725
Train Recall : 0.9266

Val Loss : 0.1909
Val Accuracy : 0.9866
Val Precision : 0.7753
Val Recall : 0.8024
Val F1 : 0.7814
Val Dice : 0.7814
Val IoU : 0.6568

No Improvement: 4/15

Epoch 51/150

Progress: 100.00% | Loss: 0.1088

Train Loss : 0.0921
Train Accuracy : 0.9932
Train Precision : 0.8691
Train Recall : 0.9282

Val Loss : 0.1703
Val Accuracy : 0.9871
Val Precision : 0.7740
Val Recall : 0.8430
Val F1 : 0.8013
Val Dice : 0.8013
Val IoU : 0.6769

Model Saved!

Epoch 52/150

Progress: 100.00% | Loss: 0.8000

Train Loss : 0.0933
Train Accuracy : 0.9932
Train Precision : 0.8695
Train Recall : 0.9269

Val Loss : 0.1759
Val Accuracy : 0.9870

Val Precision : 0.7735
Val Recall : 0.8355
Val F1 : 0.7958
Val Dice : 0.7958
Val IoU : 0.6713

No Improvement: 1/15

Epoch 53/150

Progress: 100.00% | Loss: 0.1069

Train Loss : 0.0889
Train Accuracy : 0.9934
Train Precision : 0.8724
Train Recall : 0.9309

Val Loss : 0.1800
Val Accuracy : 0.9867
Val Precision : 0.7619
Val Recall : 0.8380
Val F1 : 0.7895
Val Dice : 0.7895
Val IoU : 0.6633

No Improvement: 2/15

Epoch 54/150

Progress: 100.00% | Loss: 0.1193

Train Loss : 0.0895
Train Accuracy : 0.9935
Train Precision : 0.8749
Train Recall : 0.9312

Val Loss : 0.1798
Val Accuracy : 0.9874
Val Precision : 0.7891
Val Recall : 0.8137
Val F1 : 0.7961
Val Dice : 0.7961
Val IoU : 0.6730

No Improvement: 3/15

Epoch 55/150

Progress: 100.00% | Loss: 0.0565

Train Loss : 0.0893
Train Accuracy : 0.9933
Train Precision : 0.8707
Train Recall : 0.9302

Val Loss : 0.1767

Val Accuracy : 0.9873
Val Precision : 0.7937
Val Recall : 0.8218
Val F1 : 0.7982
Val Dice : 0.7982
Val IoU : 0.6727

No Improvement: 4/15

Epoch 56/150

Progress: 100.00% | Loss: 0.0549

Train Loss : 0.0892
Train Accuracy : 0.9935
Train Precision : 0.8751
Train Recall : 0.9306

Val Loss : 0.1760
Val Accuracy : 0.9875
Val Precision : 0.7890
Val Recall : 0.8204
Val F1 : 0.7982
Val Dice : 0.7982
Val IoU : 0.6753

No Improvement: 5/15

Epoch 57/150

Progress: 100.00% | Loss: 0.0911

Train Loss : 0.0856
Train Accuracy : 0.9936
Train Precision : 0.8763
Train Recall : 0.9333

Val Loss : 0.1819
Val Accuracy : 0.9874
Val Precision : 0.7840
Val Recall : 0.8129
Val F1 : 0.7915
Val Dice : 0.7915
Val IoU : 0.6692

No Improvement: 6/15

Epoch 58/150

Progress: 100.00% | Loss: 0.0651

Train Loss : 0.0799
Train Accuracy : 0.9941
Train Precision : 0.8850
Train Recall : 0.9383

Val Loss : 0.1698
Val Accuracy : 0.9880
Val Precision : 0.7931
Val Recall : 0.8356
Val F1 : 0.8049
Val Dice : 0.8049
Val IoU : 0.6821

Model Saved!

Epoch 59/150

Progress: 100.00% | Loss: 0.0748

Train Loss : 0.0767
Train Accuracy : 0.9942
Train Precision : 0.8873
Train Recall : 0.9395

Val Loss : 0.1814
Val Accuracy : 0.9874
Val Precision : 0.7926
Val Recall : 0.8120
Val F1 : 0.7951
Val Dice : 0.7951
Val IoU : 0.6737

No Improvement: 1/15

Epoch 60/150

Progress: 100.00% | Loss: 0.0689

Train Loss : 0.0752
Train Accuracy : 0.9944
Train Precision : 0.8904
Train Recall : 0.9419

Val Loss : 0.1754
Val Accuracy : 0.9875
Val Precision : 0.7911
Val Recall : 0.8270
Val F1 : 0.7988
Val Dice : 0.7988
Val IoU : 0.6780

No Improvement: 2/15

Epoch 61/150

Progress: 100.00% | Loss: 0.0987

Train Loss : 0.0733
Train Accuracy : 0.9945
Train Precision : 0.8903
Train Recall : 0.9443

Val Loss : 0.1826
Val Accuracy : 0.9877
Val Precision : 0.8046
Val Recall : 0.8034
Val F1 : 0.7964
Val Dice : 0.7964
Val IoU : 0.6751

No Improvement: 3/15

Epoch 62/150

Progress: 100.00% | Loss: 0.0706

Train Loss : 0.0762
Train Accuracy : 0.9946
Train Precision : 0.8929
Train Recall : 0.9442

Val Loss : 0.1673
Val Accuracy : 0.9883
Val Precision : 0.8019
Val Recall : 0.8286
Val F1 : 0.8096
Val Dice : 0.8096
Val IoU : 0.6910

Model Saved!

Epoch 63/150

Progress: 100.00% | Loss: 0.0691

Train Loss : 0.0715
Train Accuracy : 0.9946
Train Precision : 0.8936
Train Recall : 0.9456

Val Loss : 0.1706
Val Accuracy : 0.9873
Val Precision : 0.7796
Val Recall : 0.8487
Val F1 : 0.8027
Val Dice : 0.8027
Val IoU : 0.6801

No Improvement: 1/15

Epoch 64/150

Progress: 100.00% | Loss: 0.0435

Train Loss : 0.0730
Train Accuracy : 0.9947
Train Precision : 0.8948

Train Recall : 0.9459

Val Loss : 0.1698

Val Accuracy : 0.9884

Val Precision : 0.8077

Val Recall : 0.8193

Val F1 : 0.8084

Val Dice : 0.8084

Val IoU : 0.6887

No Improvement: 2/15

Epoch 65/150

Progress: 100.00% | Loss: 0.0740

Train Loss : 0.0703

Train Accuracy : 0.9948

Train Precision : 0.8959

Train Recall : 0.9468

Val Loss : 0.1697

Val Accuracy : 0.9883

Val Precision : 0.7915

Val Recall : 0.8329

Val F1 : 0.8056

Val Dice : 0.8056

Val IoU : 0.6864

No Improvement: 3/15

Epoch 66/150

Progress: 100.00% | Loss: 0.0610

Train Loss : 0.0722

Train Accuracy : 0.9948

Train Precision : 0.8962

Train Recall : 0.9471

Val Loss : 0.1662

Val Accuracy : 0.9878

Val Precision : 0.7942

Val Recall : 0.8452

Val F1 : 0.8099

Val Dice : 0.8099

Val IoU : 0.6908

Model Saved!

Epoch 67/150

Progress: 100.00% | Loss: 0.1073

Train Loss : 0.0690

Train Accuracy : 0.9948

Train Precision : 0.8967
Train Recall : 0.9474

Val Loss : 0.1700
Val Accuracy : 0.9882
Val Precision : 0.8031
Val Recall : 0.8280
Val F1 : 0.8080
Val Dice : 0.8080
Val IoU : 0.6873

No Improvement: 1/15

Epoch 68/150

Progress: 100.00% | Loss: 0.0669

Train Loss : 0.0696
Train Accuracy : 0.9949
Train Precision : 0.8983
Train Recall : 0.9487

Val Loss : 0.1734
Val Accuracy : 0.9879
Val Precision : 0.7984
Val Recall : 0.8254
Val F1 : 0.8040
Val Dice : 0.8040
Val IoU : 0.6841

No Improvement: 2/15

Epoch 69/150

Progress: 100.00% | Loss: 0.1006

Train Loss : 0.0682
Train Accuracy : 0.9949
Train Precision : 0.8985
Train Recall : 0.9490

Val Loss : 0.1720
Val Accuracy : 0.9879
Val Precision : 0.8026
Val Recall : 0.8283
Val F1 : 0.8053
Val Dice : 0.8053
Val IoU : 0.6851

No Improvement: 3/15

Epoch 70/150

Progress: 100.00% | Loss: 0.8005

Train Loss : 0.0681

Train Accuracy : 0.9950
Train Precision : 0.9000
Train Recall : 0.9504

Val Loss : 0.1772
Val Accuracy : 0.9874
Val Precision : 0.7846
Val Recall : 0.8321
Val F1 : 0.7977
Val Dice : 0.7977
Val IoU : 0.6748

No Improvement: 4/15

Epoch 71/150

Progress: 100.00% | Loss: 0.0576

Train Loss : 0.0656
Train Accuracy : 0.9951
Train Precision : 0.9006
Train Recall : 0.9507

Val Loss : 0.1643
Val Accuracy : 0.9883
Val Precision : 0.8105
Val Recall : 0.8342
Val F1 : 0.8155
Val Dice : 0.8155
Val IoU : 0.6959

Model Saved!

Epoch 72/150

Progress: 100.00% | Loss: 0.0982

Train Loss : 0.0666
Train Accuracy : 0.9950
Train Precision : 0.9007
Train Recall : 0.9497

Val Loss : 0.1746
Val Accuracy : 0.9876
Val Precision : 0.7893
Val Recall : 0.8325
Val F1 : 0.8028
Val Dice : 0.8028
Val IoU : 0.6832

No Improvement: 1/15

Epoch 73/150

Progress: 100.00% | Loss: 0.0705

Train Loss : 0.0654

Train Accuracy : 0.9951
Train Precision : 0.9017
Train Recall : 0.9515

Val Loss : 0.1707
Val Accuracy : 0.9877
Val Precision : 0.7926
Val Recall : 0.8381
Val F1 : 0.8060
Val Dice : 0.8060
Val IoU : 0.6846

No Improvement: 2/15

Epoch 74/150

Progress: 100.00% | Loss: 0.0455

Train Loss : 0.0644
Train Accuracy : 0.9951
Train Precision : 0.9020
Train Recall : 0.9517

Val Loss : 0.1686
Val Accuracy : 0.9880
Val Precision : 0.7937
Val Recall : 0.8372
Val F1 : 0.8079
Val Dice : 0.8079
Val IoU : 0.6891

No Improvement: 3/15

Epoch 75/150

Progress: 100.00% | Loss: 0.0306

Train Loss : 0.0652
Train Accuracy : 0.9952
Train Precision : 0.9023
Train Recall : 0.9524

Val Loss : 0.1763
Val Accuracy : 0.9878
Val Precision : 0.8035
Val Recall : 0.8209
Val F1 : 0.8034
Val Dice : 0.8034
Val IoU : 0.6819

No Improvement: 4/15

Epoch 76/150

Progress: 100.00% | Loss: 0.0446

Train Loss : 0.0636
Train Accuracy : 0.9952
Train Precision : 0.9029
Train Recall : 0.9530

Val Loss : 0.1613
Val Accuracy : 0.9886
Val Precision : 0.8120
Val Recall : 0.8368
Val F1 : 0.8186
Val Dice : 0.8186
Val IoU : 0.6993

Model Saved!

Epoch 77/150

Progress: 100.00% | Loss: 0.0503

Train Loss : 0.0646
Train Accuracy : 0.9953
Train Precision : 0.9052
Train Recall : 0.9524

Val Loss : 0.1680
Val Accuracy : 0.9881
Val Precision : 0.7980
Val Recall : 0.8387
Val F1 : 0.8094
Val Dice : 0.8094
Val IoU : 0.6889

No Improvement: 1/15

Epoch 78/150

Progress: 100.00% | Loss: 0.0458

Train Loss : 0.0636
Train Accuracy : 0.9953
Train Precision : 0.9048
Train Recall : 0.9537

Val Loss : 0.1742
Val Accuracy : 0.9883
Val Precision : 0.8112
Val Recall : 0.8146
Val F1 : 0.8065
Val Dice : 0.8065
Val IoU : 0.6881

No Improvement: 2/15

Epoch 79/150

Progress: 100.00% | Loss: 0.0326

Train Loss : 0.0621

Train Accuracy : 0.9954

Train Precision : 0.9064

Train Recall : 0.9536

Val Loss : 0.1824

Val Accuracy : 0.9877

Val Precision : 0.7872

Val Recall : 0.8177

Val F1 : 0.7936

Val Dice : 0.7936

Val IoU : 0.6728

No Improvement: 3/15

Epoch 80/150

Progress: 100.00% | Loss: 0.1049

Train Loss : 0.0621

Train Accuracy : 0.9953

Train Precision : 0.9060

Train Recall : 0.9536

Val Loss : 0.1813

Val Accuracy : 0.9876

Val Precision : 0.7916

Val Recall : 0.8203

Val F1 : 0.7977

Val Dice : 0.7977

Val IoU : 0.6772

No Improvement: 4/15

Epoch 81/150

Progress: 100.00% | Loss: 0.0928

Train Loss : 0.0610

Train Accuracy : 0.9954

Train Precision : 0.9061

Train Recall : 0.9547

Val Loss : 0.1743

Val Accuracy : 0.9876

Val Precision : 0.7895

Val Recall : 0.8329

Val F1 : 0.8034

Val Dice : 0.8034

Val IoU : 0.6817

No Improvement: 5/15

Epoch 82/150
Progress: 100.00% | Loss: 0.0485
Train Loss : 0.0616
Train Accuracy : 0.9954
Train Precision : 0.9066
Train Recall : 0.9543

Val Loss : 0.1764
Val Accuracy : 0.9880
Val Precision : 0.7863
Val Recall : 0.8269
Val F1 : 0.7994
Val Dice : 0.7994
Val IoU : 0.6785

No Improvement: 6/15

Epoch 83/150
Progress: 100.00% | Loss: 0.8000
Train Loss : 0.0607
Train Accuracy : 0.9956
Train Precision : 0.9100
Train Recall : 0.9561

Val Loss : 0.1664
Val Accuracy : 0.9885
Val Precision : 0.7951
Val Recall : 0.8394
Val F1 : 0.8104
Val Dice : 0.8104
Val IoU : 0.6924

No Improvement: 7/15

Epoch 84/150
Progress: 100.00% | Loss: 0.0614
Train Loss : 0.0581
Train Accuracy : 0.9956
Train Precision : 0.9114
Train Recall : 0.9566

Val Loss : 0.1648
Val Accuracy : 0.9882
Val Precision : 0.8044
Val Recall : 0.8444
Val F1 : 0.8156
Val Dice : 0.8156
Val IoU : 0.6984

No Improvement: 8/15

Epoch 85/150

Progress: 100.00% | Loss: 0.8000

Train Loss : 0.0599

Train Accuracy : 0.9957

Train Precision : 0.9116

Train Recall : 0.9573

Val Loss : 0.1681

Val Accuracy : 0.9886

Val Precision : 0.8088

Val Recall : 0.8338

Val F1 : 0.8130

Val Dice : 0.8130

Val IoU : 0.6935

No Improvement: 9/15

Epoch 86/150

Progress: 100.00% | Loss: 0.1199

Train Loss : 0.0579

Train Accuracy : 0.9957

Train Precision : 0.9128

Train Recall : 0.9576

Val Loss : 0.1717

Val Accuracy : 0.9882

Val Precision : 0.7959

Val Recall : 0.8318

Val F1 : 0.8065

Val Dice : 0.8065

Val IoU : 0.6893

No Improvement: 10/15

Epoch 87/150

Progress: 100.00% | Loss: 0.0743

Train Loss : 0.0618

Train Accuracy : 0.9958

Train Precision : 0.9136

Train Recall : 0.9585

Val Loss : 0.1724

Val Accuracy : 0.9882

Val Precision : 0.8070

Val Recall : 0.8277

Val F1 : 0.8075

Val Dice : 0.8075

Val IoU : 0.6885

No Improvement: 11/15

Epoch 88/150

Progress: 100.00% | Loss: 0.0491

Train Loss : 0.0577

Train Accuracy : 0.9958

Train Precision : 0.9137

Train Recall : 0.9595

Val Loss : 0.1675

Val Accuracy : 0.9887

Val Precision : 0.8185

Val Recall : 0.8291

Val F1 : 0.8146

Val Dice : 0.8146

Val IoU : 0.6977

No Improvement: 12/15

Epoch 89/150

Progress: 100.00% | Loss: 0.0548

Train Loss : 0.0568

Train Accuracy : 0.9959

Train Precision : 0.9163

Train Recall : 0.9592

Val Loss : 0.1644

Val Accuracy : 0.9885

Val Precision : 0.8000

Val Recall : 0.8449

Val F1 : 0.8133

Val Dice : 0.8133

Val IoU : 0.6960

No Improvement: 13/15

Epoch 90/150

Progress: 100.00% | Loss: 0.0888

Train Loss : 0.0562

Train Accuracy : 0.9959

Train Precision : 0.9160

Train Recall : 0.9609

Val Loss : 0.1669

Val Accuracy : 0.9884

Val Precision : 0.7994

Val Recall : 0.8413

Val F1 : 0.8127

Val Dice : 0.8127

Val IoU : 0.6948

No Improvement: 14/15

Epoch 91/150

Progress: 100.00% | Loss: 0.0430

Train Loss : 0.0548

Train Accuracy : 0.9959

Train Precision : 0.9160

Train Recall : 0.9605

Val Loss : 0.1700

Val Accuracy : 0.9884

Val Precision : 0.7998

Val Recall : 0.8323

Val F1 : 0.8088

Val Dice : 0.8088

Val IoU : 0.6910

No Improvement: 15/15

Early Stopping Triggered after Epoch 91

```
# =====
```

```
# LOAD BEST MODEL
```

```
# =====
```

```
model.load_state_dict(  
    torch.load(  
        "segformer_best.pth",  
        map_location=device  
    )  
)
```

```
model.eval()
```

```
# =====
```

```
# METRIC VARIABLES
```

```
# =====
```

```
acc = 0
```

```
prec = 0
```

```
rec = 0
```

```
f1_total = 0
```

```

dice_total = 0
iou_total = 0
count = 0
all_preds = []
all_targets = []

# =====
# TEST EVALUATION
# =====

with torch.no_grad():
    for images, masks in test_loader:
        images = images.to(device)
        masks = masks.to(device)
        pred = model(images)
        pred = torch.sigmoid(pred)
        a, p, r, f1, d, i = calculate_all_metrics(
            pred,
            masks
        )
        acc += a
        prec += p
        rec += r
        f1_total += f1
        dice_total += d
        iou_total += i
        count += 1
        pred_bin = (
            pred > 0.5

```

```

        ).cpu().numpy().flatten()
    target_bin = (
        masks
        ).cpu().numpy().flatten()
    all_preds.extend(
        pred_bin
    )
    all_targets.extend(
        target_bin
    )

# =====
# FINAL RESULTS
# =====

print("\nFINAL TEST RESULTS\n")
print("Test Accuracy :", acc / count)
print("Test Precision:", prec / count)
print("Test Recall   :", rec / count)
print("Test F1 Score  :", f1_total / count)
print("Test Dice      :", dice_total / count)
print("Test IoU       :", iou_total / count)

# =====
# CONFUSION MATRIX
# =====

cm = confusion_matrix(
    all_targets,
    all_preds
)

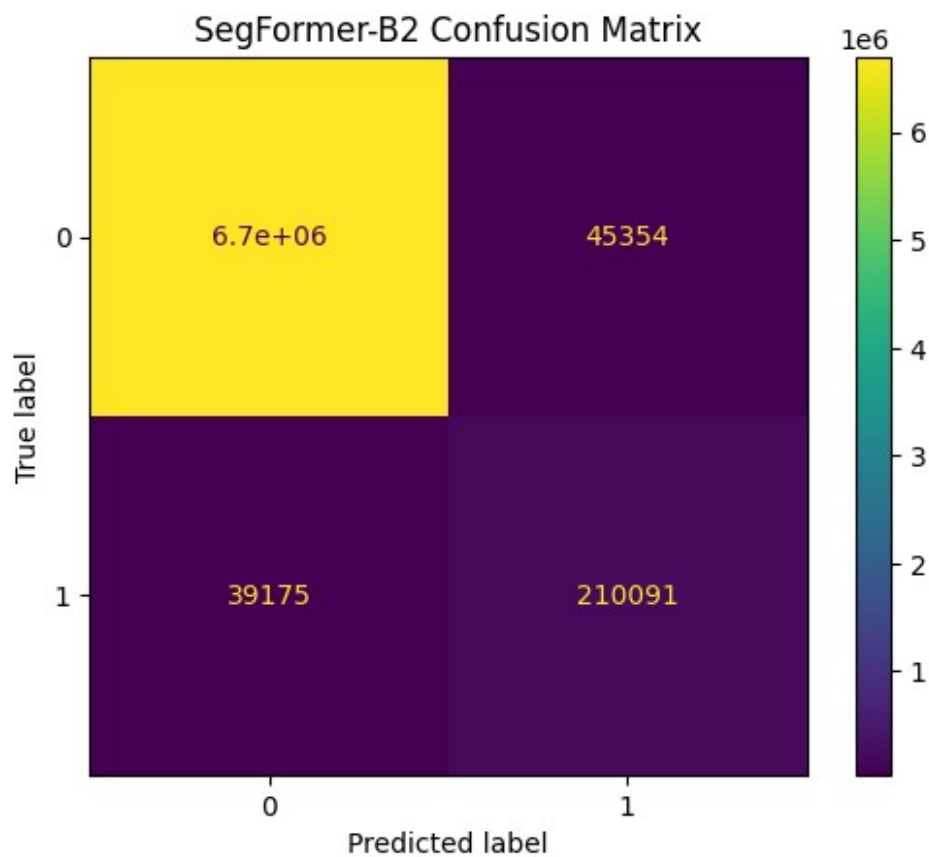
```



```
ConfusionMatrixDisplay(
    cm
).plot()
plt.title(
    "SegFormer-B2 Confusion Matrix"
)
plt.show()
```

FINAL TEST RESULTS

```
Test Accuracy : 0.987952084703879
Test Precision: 0.8046185018664057
Test Recall   : 0.8368936838074164
Test F1 Score : 0.8140768595039845
Test Dice     : 0.8140768595039845
Test IoU      : 0.6944032890552824
```



```

# ===== VALIDATION EVALUATION =====

model.eval()

acc = 0
prec = 0
rec = 0
f1_total = 0
dice_total = 0
iou_total = 0

count = 0

for x, y in val_loader:

    x = x.to(device)
    y = y.to(device)

    with torch.no_grad():

        pred = model(x)

        pred = torch.sigmoid(pred)

        a, p, r, f1, d, i = calculate_all_metrics(
            pred,
            y
        )

        acc += a
        prec += p
        rec += r
        f1_total += f1
        dice_total += d
        iou_total += i

        count += 1

print("\nVALIDATION RESULTS\n")

print("Final Validation Accuracy:", acc/count)

print("Final Validation Precision:", prec/count)

print("Final Validation Recall    :", rec/count)

print("Final Validation F1 Score :", f1_total/count)

print("Final Validation Dice      :", dice_total/count)

print("Final Validation IoU       :", iou_total/count)

```

VALIDATION RESULTS

Final Validation Accuracy: 0.9885176128961823
Final Validation Precision: 0.8113658698275685
Final Validation Recall : 0.8142008881338618
Final Validation F1 Score : 0.8090254868465391
Final Validation Dice : 0.8090254868465391
Final Validation IoU : 0.690678643020378

```
epochs_range = range(
    1,
    len(train_losses)+1
)

plt.figure(figsize=(15,10))

# ===== LOSS =====

plt.subplot(2,2,1)

plt.plot(
    epochs_range,
    train_losses,
    marker='o'
)

plt.title(
    "Loss vs Epoch"
)

plt.grid()

# ===== ACCURACY =====

plt.subplot(2,2,2)

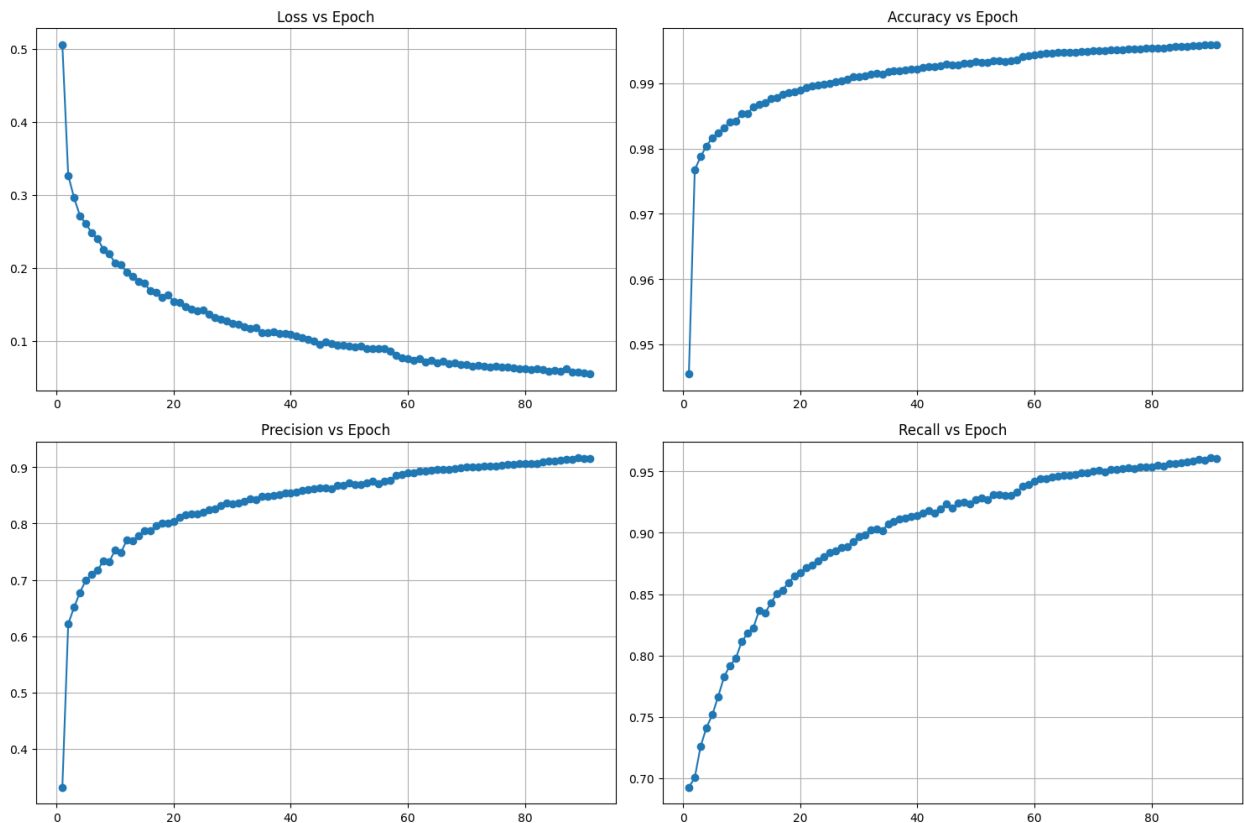
plt.plot(
    epochs_range,
    train_acc,
```

```

        marker='o'
    )
plt.title(
    "Accuracy vs Epoch"
)
plt.grid()
# ===== PRECISION =====
plt.subplot(2,2,3)
plt.plot(
    epochs_range,
    train_prec,
    marker='o'
)
plt.title(
    "Precision vs Epoch"
)
plt.grid()
# ===== RECALL =====
plt.subplot(2,2,4)
plt.plot(
    epochs_range,
    train_rec,
    marker='o'
)
plt.title(
    "Recall vs Epoch"
)

```

```
)
plt.grid()
plt.tight_layout()
plt.show()
```



```
import matplotlib.pyplot as plt

epochs_range = range(
    1,
    len(train_acc) + 1
)

# =====
# ACCURACY
# =====

plt.figure(figsize=(10,6))
plt.plot(
```

```
        epochs_range,
        train_acc,
        label="Train Accuracy"
    )
plt.plot(
    epochs_range,
    val_acc,
    label="Validation Accuracy"
)
plt.title("Accuracy vs Epoch")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.show()

# =====
# PRECISION
# =====

plt.figure(figsize=(10,6))
plt.plot(
    epochs_range,
    train_prec,
    label="Train Precision"
)
plt.plot(
    epochs_range,
    val_prec,
```

```

        label="Validation Precision"
    )
plt.title("Precision vs Epoch")
plt.xlabel("Epoch")
plt.ylabel("Precision")
plt.legend()
plt.grid(True)
plt.show()

# =====
# RECALL
# =====

plt.figure(figsize=(10,6))
plt.plot(
    epochs_range,
    train_rec,
    label="Train Recall"
)
plt.plot(
    epochs_range,
    val_rec,
    label="Validation Recall"
)
plt.title("Recall vs Epoch")
plt.xlabel("Epoch")
plt.ylabel("Recall")
plt.legend()
plt.grid(True)

```

```
plt.show()

# =====
# VALIDATION F1 SCORE
# =====

plt.figure(figsize=(10,6))
plt.plot(
    epochs_range,
    val_f1,
    label="Validation F1 Score"
)
plt.title("Validation F1 Score vs Epoch")
plt.xlabel("Epoch")
plt.ylabel("F1 Score")
plt.legend()
plt.grid(True)
plt.show()

# =====
# VALIDATION DICE
# =====

plt.figure(figsize=(10,6))
plt.plot(
    epochs_range,
    val_dice,
    label="Validation Dice"
)
plt.title("Validation Dice vs Epoch")
plt.xlabel("Epoch")
```



```

plt.ylabel("Dice")
plt.legend()
plt.grid(True)
plt.show()

# =====
# VALIDATION IoU
# =====

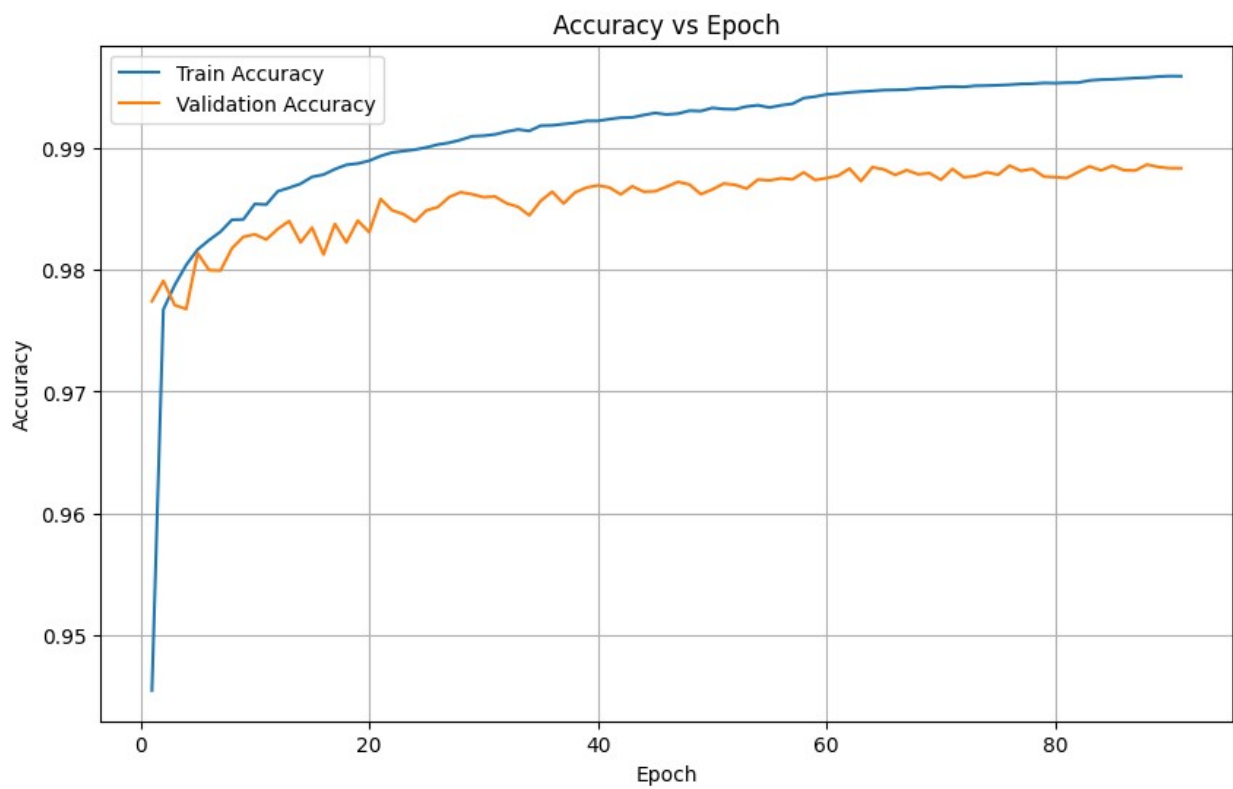
plt.figure(figsize=(10,6))
plt.plot(
    epochs_range,
    val_iou,
    label="Validation IoU"
)
plt.title("Validation IoU vs Epoch")
plt.xlabel("Epoch")
plt.ylabel("IoU")
plt.legend()
plt.grid(True)
plt.show()

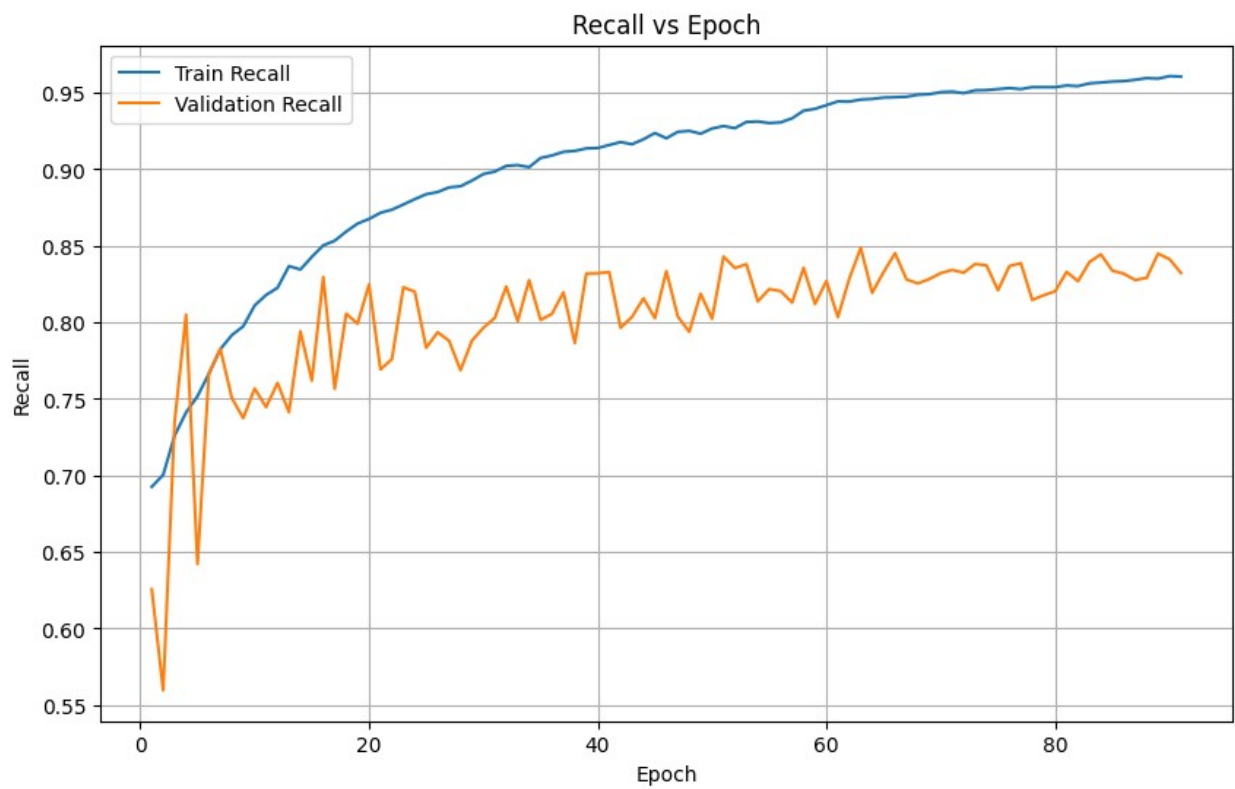
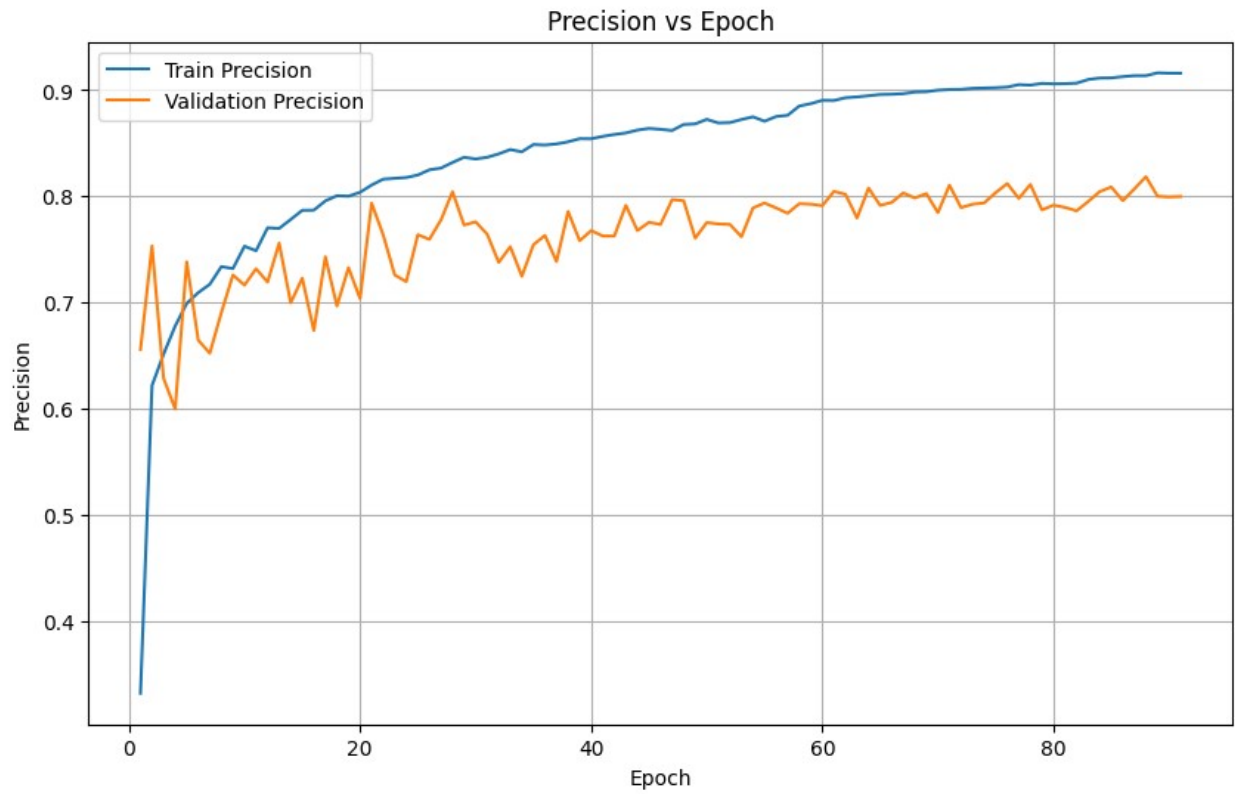
# =====
# LOSS
# =====

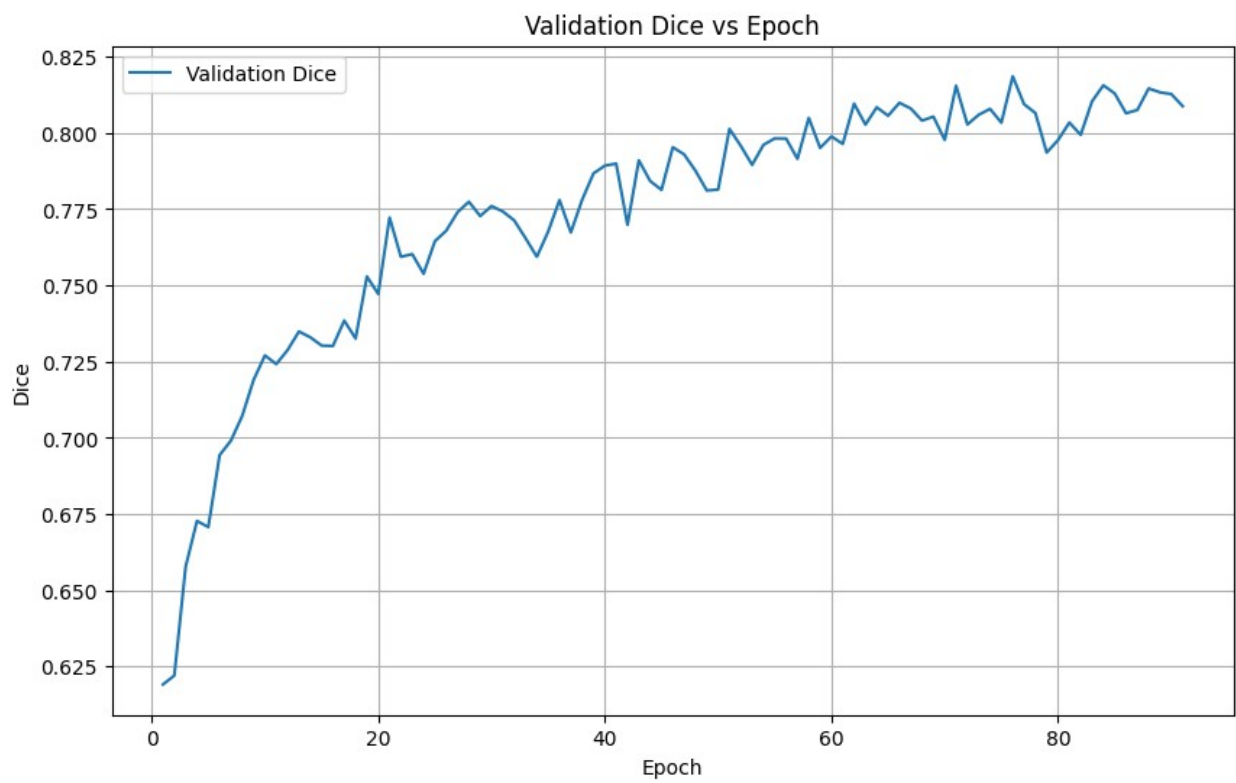
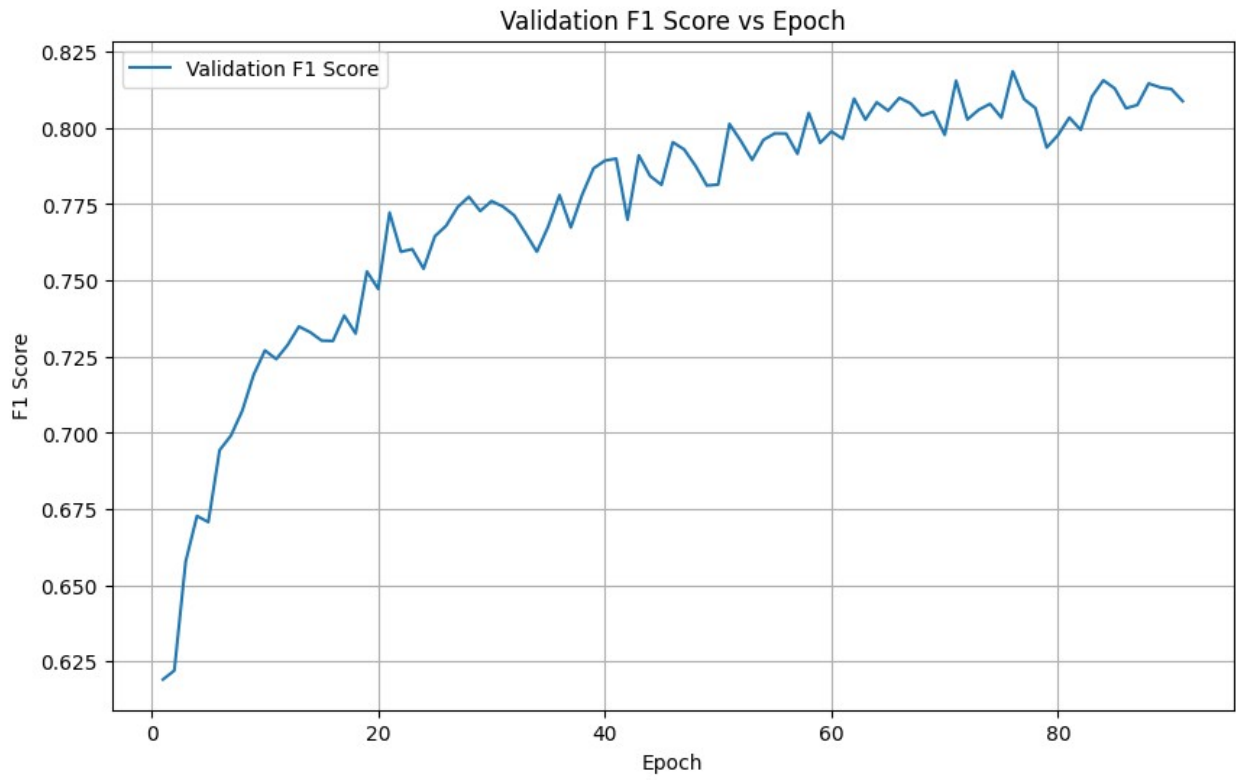
plt.figure(figsize=(10,6))
plt.plot(
    epochs_range,
    train_losses,
    label="Train Loss"
)

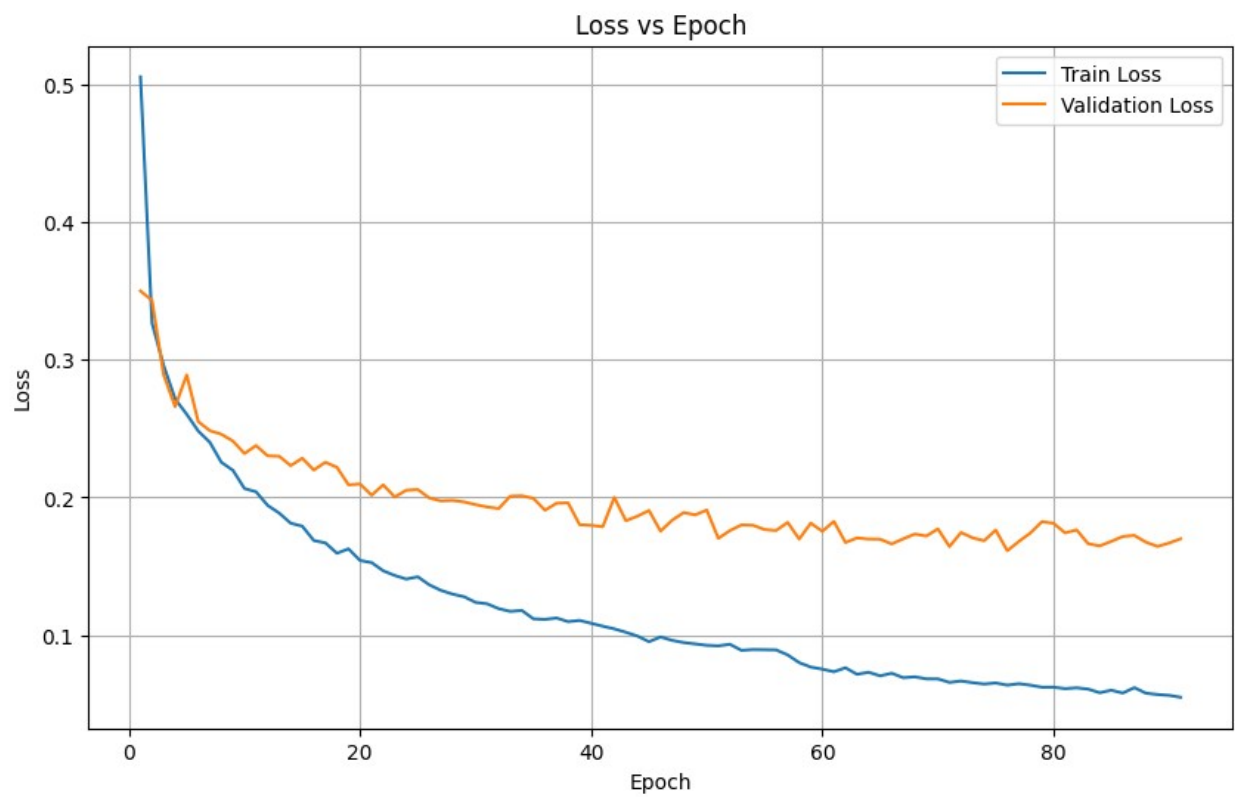
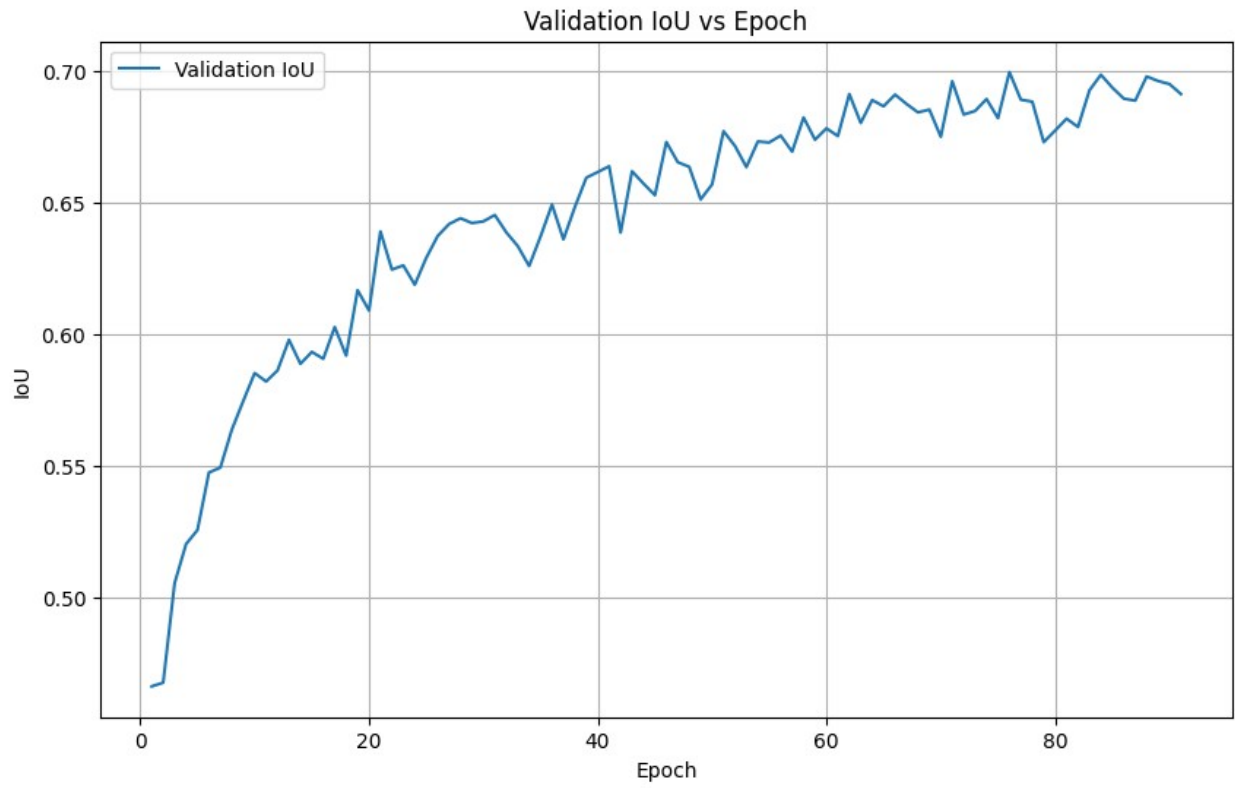
```

```
plt.plot(
    epochs_range,
    val_losses,
    label="Validation Loss"
)
plt.title("Loss vs Epoch")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.show()
```









```
model.eval()
images_collected = 0
num_images = 10
all_inputs = []
all_masks = []
all_preds = []
for images, masks in test_loader:
    images = images.to(device)
    with torch.no_grad():
        pred = model(images)
        pred = torch.sigmoid(pred)
    all_inputs.extend(
        images.cpu().numpy()
    )
    all_masks.extend(
        masks.numpy()
    )
    all_preds.extend(
        pred.cpu().numpy()
    )
    images_collected += images.shape[0]
    if images_collected >= num_images:
        break
all_inputs = np.array(all_inputs)
all_masks = np.array(all_masks)
all_preds = np.array(all_preds)
```

```
plt.figure(figsize=(20,30))
for i in range(num_images):
    input_img = all_inputs[i][0]
    true_mask = all_masks[i][0]
    pred_mask = all_preds[i][0]
    binary_pred = (
        pred_mask > 0.5
    ).astype(float)
    heatmap = pred_mask
    overlay = (
        0.6 * input_img
        +
        0.4 * heatmap
    )
    # INPUT
    plt.subplot(
        num_images,
        5,
        i*5 + 1
    )
    plt.title("Input")
    plt.imshow(
        input_img,
        cmap='gray'
    )
    plt.axis('off')
```

```
# GROUND TRUTH

plt.subplot(
    num_images,
    5,
    i*5 + 2
)

plt.title("Ground Truth")

plt.imshow(
    true_mask,
    cmap='gray'
)

plt.axis('off')

# PREDICTION

plt.subplot(
    num_images,
    5,
    i*5 + 3
)

plt.title("Prediction")

plt.imshow(
    binary_pred,
    cmap='gray'
)

plt.axis('off')

# HEATMAP

plt.subplot(
```



```
        num_images,
        5,
        i*5 + 4
    )
plt.title("Heatmap")
plt.imshow(
    heatmap,
    cmap='jet'
)
plt.axis('off')
# OVERLAY
plt.subplot(
    num_images,
    5,
    i*5 + 5
)
plt.title("Overlay")
plt.imshow(
    overlay,
    cmap='jet'
)
plt.axis('off')
plt.tight_layout()
plt.show()
```

